

Physique dans le jeu vidéo

Table des matières

Session 1 : Géométrie et Calcul Vectoriel	2
Session 2 : Cinématique 2D et Mouvement de Projectile	7
Position en 2D	8
Vitesse en 2D	9
Accélération en 2D	11
Session 3 : Les lois de Newton et la méthode Euler	14
Objectifs de la Session :	14
Les 3 Lois de Newton	14
Intégration Numérique & Série de Taylor	17
Session 4 : Les Forces de Frottement	19
TP : Le Laboratoire de Chocs 1D	20
Annexe : L'Impulsion de Collision	23
Exercice : Frottement sur Plan Incliné	25
Session 5 : Cahier d'Exercices - Vecteurs & Physique	28
Corrigé : Vecteurs & Physique	31
Session 6 : Broad & Narrow Phases	35
La Broad Phase	37
La Narrow Phase	37
Session 7 : Révisions et Quiz	39
Session 8 : Physique de la Rotation	40
Étape 1 : Mise à jour de la classe Rigidbody	43
Étape 2 : La méthode ApplyForce(force, worldPoint)	43
Étape 3 : L'intégrateur (Euler Semi-Implicite)	43
Session 9 : L'Intégration de Verlet	44
Annexe : La Méthode Runge-Kutta (RK4)	45
Session 10 : Encore plus de forces	48
Session 11 : Moteurs Physiques & Middleware	50
Session 12 : RigidBodyes & Colliders	53
Session 13 : Joints, Moteurs & Raycasts	55
Session 14 : Les Solveurs de Contraintes	57
Session 15 : Cinématique Inverse (IK)	59
Session 16 : Les Systèmes de Particules et le GPU	61
Session 17 : Character Controllers	63
Session 18 : Physique des Véhicules	65
Session 19 : Physique & IA	67
TP Guide : Unity & IA	69

Session 1 : Géométrie et Calcul Vectoriel

Définitions

Un vecteur est une grandeur ayant une direction et une magnitude.

On dit qu'il est de dimension n si il a n composantes.

Dans le plan, $n = 2$, donc un vecteur a 2 composantes, 2 dimensions.

🔗 Représentation

Un vecteur est représenté par une flèche. Il peut être placé n'importe où dans le plan. On le place habituellement par rapport au contexte de ce qu'il représente.



Fig. 1. – Un vecteur

💡 Notation vectorielle

Même si le vecteur est défini par ses magnitude et direction, on utilise une notation vectorielle, qui décompose le vecteur selon les vecteurs $\hat{i}$ et $\hat{j}$ du plan cartésien. On retrouve ses 2 dimensions représentées en colonne.

Exemple.

$$\vec{v} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2\hat{i} + 3\hat{j}$$

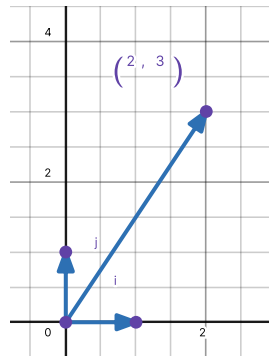


Fig. 2. – Le vecteur $\vec{v}$

🔗 Déplacement

Un déplacement peut se décrire par le vecteur reliant le point A (départ) au point B (arrivée) :

$$\overrightarrow{AB} = \begin{pmatrix} x_B - x_A \\ y_B - y_A \end{pmatrix}$$

🔗 Des vecteurs célèbres en physique

- **Le vecteur position**

Souvent noté $\vec{r}$ (radius), représente la position d'un point dans l'espace en partant de l'origine.

- **Le vecteur vitesse**

Noté $\vec{v}$, représente la vitesse d'un objet, souvent représenté partant du centre de l'objet.

- **Le vecteur accélération**

Noté $\vec{a}$, représente l'accélération d'un objet, souvent représenté partant du centre de l'objet.

Magnitude et direction

La longueur du vecteur, aussi appelée magnitude, est notée $\|\vec{v}\|$, et parfois simplement $|\vec{v}|$.

En 2D : On utilise le théorème de Pythagore pour calculer la norme (autre nom de la magnitude) :

$$\|\vec{v}\| = \sqrt{v_x^2 + v_y^2}$$

Vecteur Unitaire ($\hat{u}$) : Vecteur de longueur 1 direction $\vec{v}$:

$$\hat{u} = \frac{\vec{v}}{\|\vec{v}\|}$$

Un vecteur unitaire est un vecteur de longueur 1, il est souvent utilisé pour représenter une direction sans avoir à se soucier de la magnitude.

Opérations de Base

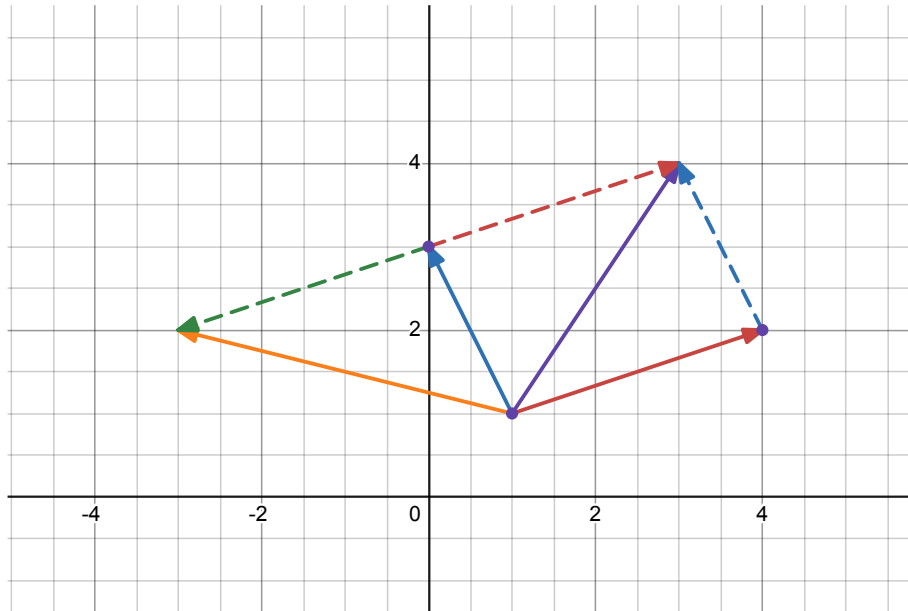
Soit $\vec{u} = \begin{pmatrix} u_x \\ u_y \end{pmatrix}$ et $\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$.

Addition :

$$\vec{u} + \vec{v} = \begin{pmatrix} u_x + v_x \\ u_y + v_y \end{pmatrix}$$

Soustraction :

$$\vec{u} - \vec{v} = \begin{pmatrix} u_x - v_x \\ u_y - v_y \end{pmatrix}$$

Fig. 3. – Le vecteur $\vec{u}$ (rouge) et le vecteur $\vec{v}$ (bleu)

Multiplication par scalaire (k) :

$$k \cdot \vec{u} = \begin{pmatrix} ku_x \\ ku_y \end{pmatrix}$$

Produit Scalaire

Résultat : un **nombre** (scalaire).

Formule algébrique :

$$\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y + u_z v_z$$

Formule géométrique :

$$\vec{u} \cdot \vec{v} = \|\vec{u}\| \cdot \|\vec{v}\| \cdot \cos(\theta)$$

1. Si $\vec{u} \cdot \vec{v} = 0$, alors $\vec{u} \perp \vec{v}$.
2. Si $\vec{u} \cdot \vec{v} > 0$, alors l'angle entre les vecteurs est inférieur à 90° . (devant)
3. Si $\vec{u} \cdot \vec{v} < 0$, alors l'angle entre les vecteurs est supérieur à 90° . (derrière)

Angle entre 2 vecteurs

On isole $\cos(\theta)$. :

$$\cos(\theta) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}$$

$$\theta = \arccos\left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|}\right)$$

Projections

Projection de $\vec{u}$ sur $\vec{v}$. On utilise le produit scalaire, dans la direction sur laquelle on projette.

$$\text{proj}_{\vec{v}}(\vec{u}) = \left(\frac{\vec{u} \cdot \vec{v}}{\|\vec{v}\|^2} \right) \cdot \vec{v}$$

Et si $\vec{v}$ est normalisé (direction de la droite sur laquelle on projète), on peut simplifier :

$$\text{proj}_{\vec{v}}(\vec{u}) = (\vec{u} \cdot \vec{v}) \cdot \vec{v}$$

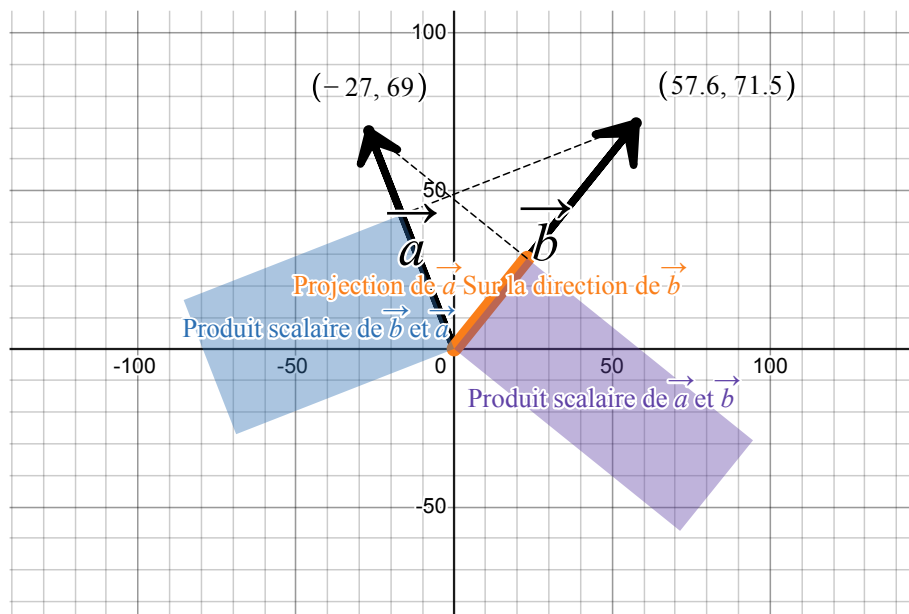


Fig. 4. – Produit scalaire, projection

Produit Vectoriel (3D)

Résultat : un **vecteur** perpendiculaire.

$$\vec{w} = \vec{a} \times \vec{b}$$

Commutativité : Attention, le produit vectoriel n'est pas commutatif ! On a un changement de signe :

$$\vec{a} \times \vec{b} = -\vec{b} \times \vec{a}$$

Magnitude :

$$\|\vec{a} \times \vec{b}\| = \|\vec{a}\| \cdot \|\vec{b}\| \cdot \sin(\theta)$$

Direction : Le produit vectoriel est perpendiculaire aux deux vecteurs :

$$\vec{a} \times \vec{b} \perp \vec{a}$$

$$\vec{a} \times \vec{b} \perp \vec{b}$$

On utilise la règle de la main droite pour trouver la direction du produit vectoriel.

- Le pouce pointe sur a

- l'index sur b
- le majeur sur $a \times b$.

Calcul (Déterminant) : Le déterminant est un outil algébrique qui permet de trouver les solutions d'un problème de n équations avec n inconnues. Cette solution représente, en quelque sorte une direction émergente (perpendiculaire) des équations.

$$a = \begin{pmatrix} a_x \\ a_y \\ a_z \end{pmatrix} \text{ et } b = \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix}$$

$$\vec{a} \times \vec{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Session 2 : Cinématique 2D et Mouvement de Projectile

🔗 Introduction à la cinématique

La cinématique est la branche de la mécanique qui décrit le mouvement des objets sans considérer les causes du mouvement (les forces). Contrairement à la dynamique, qui relie le mouvement aux forces.

Notre objectif dans ce bloc est de développer les outils mathématiques pour décrire précisément *comment* les objets se déplacent.

Position, Vitesse et Accélération

La position, la vitesse et l'accélération sont des concepts qui permettent de décrire précisément comment un objet se comporte.

- La **position** est le vecteur qui indique la localisation d'un point dans l'espace.
- La **vitesse** est le vecteur qui indique la vitesse du point, définie comme la variation (dérivée) de la position par rapport au temps.
- L'**accélération** est le vecteur qui indique l'accélération du point, définie comme la variation (dérivée) de la vitesse par rapport au temps.

🔗 Cas particuliers

- Lorsque la position est constante, le point est au repos, immobile. Sa vitesse et son accélération sont nulles.
- Lorsque la vitesse est constante, le point se déplace à vitesse constante en ligne droite. Son accélération est nulle.
- Lorsque l'accélération est constante, la vitesse augmente constamment, le point se déplace de plus en plus vite.

1. Introduction et Rappels

Rappels sur les vecteurs

Rappel de la définition d'un vecteur comme une quantité possédant une magnitude (longueur, norme) et une direction.

Représentation d'un vecteur en 2D à l'aide de ses composantes :

$$\vec{v} = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

.

Opérations vectorielles de base :

$$\vec{a} + \vec{b} = \begin{pmatrix} a_x + b_x \\ a_y + b_y \end{pmatrix}$$

$$\vec{a} - \vec{b} = \begin{pmatrix} a_x - b_x \\ a_y - b_y \end{pmatrix}$$

Multiplication par un scalaire :

$$k\vec{a} = \begin{pmatrix} ka_x \\ ka_y \end{pmatrix}$$

Position en 2D

Vecteur Position

Pour décrire la localisation d'un point dans un plan bidimensionnel, nous utilisons le **vecteur position**, noté $\vec{r}$ (ou parfois $\vec{s}$ ou $\vec{x}$).

Avec un système de coordonnées cartésiennes (x, y) , le vecteur position d'un point P de coordonnées (x, y) est :

$$\vec{r} = \begin{pmatrix} x \\ y \end{pmatrix} = x\hat{i} + y\hat{j}$$

où $\hat{i} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ et $\hat{j} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ sont les vecteurs unitaires.

- Le vecteur position pointe de l'origine vers la position de l'objet.
- La **magnitude** $|\vec{r}| = \sqrt{x^2 + y^2}$ représente la distance à l'origine.
- La **direction** est donnée par l'angle θ avec $\tan(\theta) = \frac{y}{x}$.

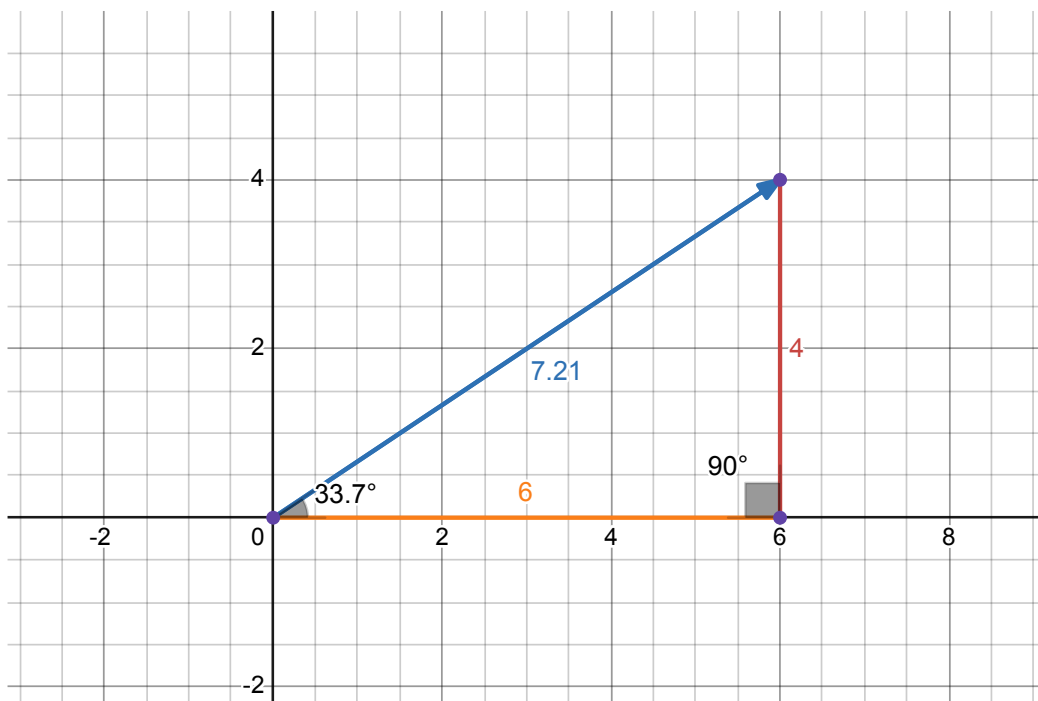


Fig. 5. – Plan 2D avec vecteurs position et vitesse

🔗 Trajectoire

Si la position d'un objet change au cours du temps, on décrit son mouvement par :

$$\vec{r}(t) = \begin{pmatrix} x(t) \\ y(t) \end{pmatrix}$$

L'ensemble des points atteints forme la **trajectoire** — une courbe dans l'espace.

Exemples de trajectoires :

- **Mouvement rectiligne uniforme** : $\vec{r}(t) = \begin{pmatrix} x_0 + v_x t \\ y_0 + v_y t \end{pmatrix}$ — ligne droite.
- **Mouvement circulaire uniforme** : $\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$ — cercle.
- **Mouvement parabolique (projectile)** : $\vec{r}(t) = \begin{pmatrix} v_{0x} t \\ y_0 + v_{0y} t - \frac{1}{2} g t^2 \end{pmatrix}$ — parabole.

Vitesse en 2D

Vitesse Moyenne

Le **déplacement** d'un objet pendant un intervalle $\Delta t = t_f - t_i$ est :

$$\Delta \vec{r} = \vec{r}_f - \vec{r}_i = \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix}$$

Le **vecteur vitesse moyenne** $\vec{v}_{\text{moy}}$ est le rapport du déplacement au temps écoulé :

$$\vec{v}_{\text{moy}} = \frac{\Delta \vec{r}}{\Delta t} = \begin{pmatrix} \frac{\Delta x}{\Delta t} \\ \frac{\Delta y}{\Delta t} \end{pmatrix} = \begin{pmatrix} v_{\text{moy},x} \\ v_{\text{moy},y} \end{pmatrix}$$

La vitesse moyenne a la même direction que le déplacement, et sa magnitude est le déplacement total divisé par le temps écoulé.

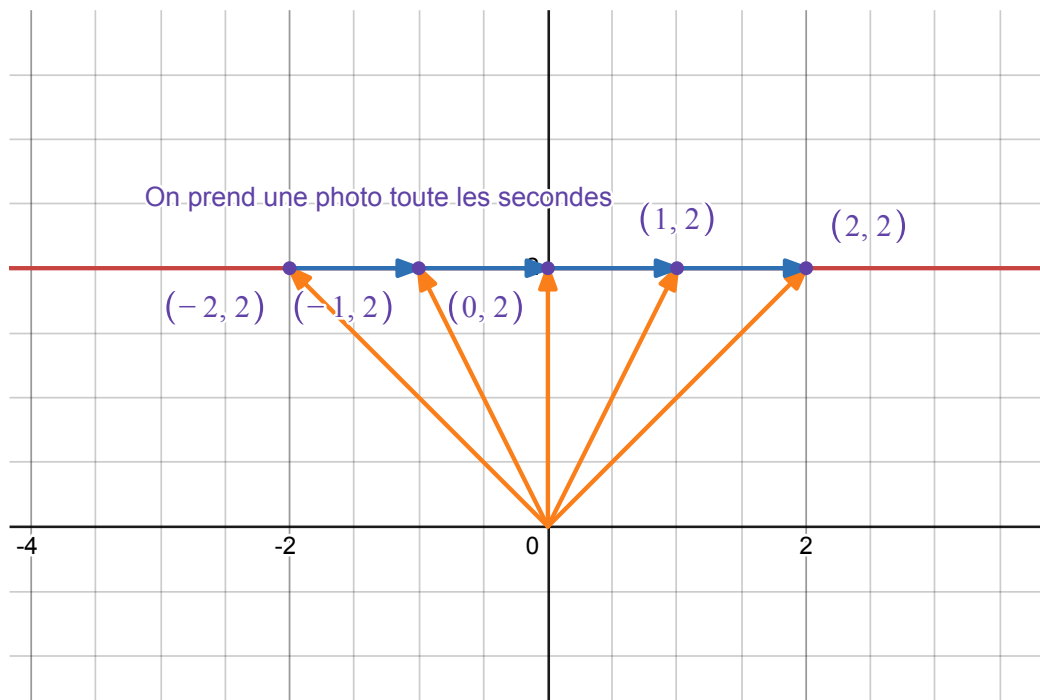


Fig. 6. – Plan 2D avec vecteurs position et vitesse

Exemple. Si une voiture se déplace de $\vec{r}_i = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ à $\vec{r}_f = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ en $\Delta t = 5$ secondes, alors le déplacement est $\Delta \vec{r} = \begin{pmatrix} 10 \\ 5 \end{pmatrix}$ et la vitesse moyenne est $\vec{v}_{\text{moy}} = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$ m/s.

Vitesse Instantanée

La **vitesse instantanée** $\vec{v}(t)$ est la limite de la vitesse moyenne lorsque $\Delta t \rightarrow 0$:

$$\vec{v}(t) = \lim_{\Delta t \rightarrow 0} \frac{\Delta \vec{r}}{\Delta t} = \frac{d\vec{r}}{dt}$$

En composantes :

$$\vec{v}(t) = \begin{pmatrix} \frac{dx(t)}{dt} \\ \frac{dy(t)}{dt} \end{pmatrix} = \begin{pmatrix} v_x(t) \\ v_y(t) \end{pmatrix}$$

- La **magnitude** $|\vec{v}(t)| = \sqrt{v_{x(t)}^2 + v_{y(t)}^2}$ est la **vitesse scalaire**.
- La **direction** est tangente à la trajectoire au point considéré.

🔗 Application aux types de trajectoire

1. Mouvement rectiligne uniforme :

$$\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

— Le vecteur vitesse est **constant**.

2. Mouvement circulaire uniforme :

$$\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$$

Magnitude constante : $|\vec{v}| = R\omega$. La vitesse est tangente au cercle.

3. Mouvement parabolique (projectile) :

$$\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - gt \end{pmatrix}$$

La composante horizontale v_x reste constante, la verticale v_y diminue linéairement.

Accélération en 2D

Accélération Moyenne

Le **vecteur accélération moyenne** $\vec{a}_{\text{moy}}$ pendant un intervalle $\Delta t = t_f - t_i$ est le rapport du changement de vitesse au temps écoulé :

$$\vec{a}_{\text{moy}} = \frac{\Delta \vec{v}}{\Delta t} = \begin{pmatrix} \frac{\Delta v_x}{\Delta t} \\ \frac{\Delta v_y}{\Delta t} \end{pmatrix} = \begin{pmatrix} a_{\text{moy},x} \\ a_{\text{moy},y} \end{pmatrix}$$

L'accélération moyenne a la direction du changement de vitesse.

Accélération Instantanée

L'**accélération instantanée** $\vec{a}(t)$ est la dérivée de la vitesse par rapport au temps :

$$\vec{a}(t) = \frac{d\vec{v}}{dt} = \begin{pmatrix} \frac{dv_x(t)}{dt} \\ \frac{dv_y(t)}{dt} \end{pmatrix} = \begin{pmatrix} \frac{d^2x(t)}{dt^2} \\ \frac{d^2y(t)}{dt^2} \end{pmatrix}$$

L'accélération peut changer la magnitude de la vitesse (accélérer/décélérer), sa direction, ou les deux.

Cas particulier : Accélération constante

Si l'accélération $\vec{a}$ est constante, on peut **intégrer** pour obtenir la vitesse et la position :

$$\vec{v}(t) = \vec{v}_0 + \vec{a}t = \begin{pmatrix} v_{0x} + a_x t \\ v_{0y} + a_y t \end{pmatrix}$$

$$\vec{r}(t) = \vec{r}_0 + \vec{v}_0 t + \frac{1}{2} \vec{a} t^2 = \begin{pmatrix} x_0 + v_{0x} t + \frac{1}{2} a_x t^2 \\ y_0 + v_{0y} t + \frac{1}{2} a_y t^2 \end{pmatrix}$$

où $\vec{v}_0 = \begin{pmatrix} v_{0x} \\ v_{0y} \end{pmatrix}$ est la vitesse initiale et $\vec{r}_0 = \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$ est la position initiale.

C'est ce cas particulier qui est crucial pour l'étude du mouvement de projectile sous l'effet de la gravité.

🔗 Exemples par type de trajectoire

1. Mouvement rectiligne uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} x_0 + v_x t \\ y_0 + v_y t \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_x \\ v_y \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$

La dérivée d'un vecteur constant est nulle. Aucune accélération n'est nécessaire pour maintenir le mouvement rectiligne uniforme.

2. Mouvement circulaire uniforme :

- Position : $\vec{r}(t) = \begin{pmatrix} R \cos(\omega t) \\ R \sin(\omega t) \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} -R\omega \sin(\omega t) \\ R\omega \cos(\omega t) \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} -R\omega^2 \cos(\omega t) \\ -R\omega^2 \sin(\omega t) \end{pmatrix}$

On observe que $\vec{a}(t) = -\omega^2 \vec{r}(t)$: l'accélération est dirigée vers le centre du cercle.

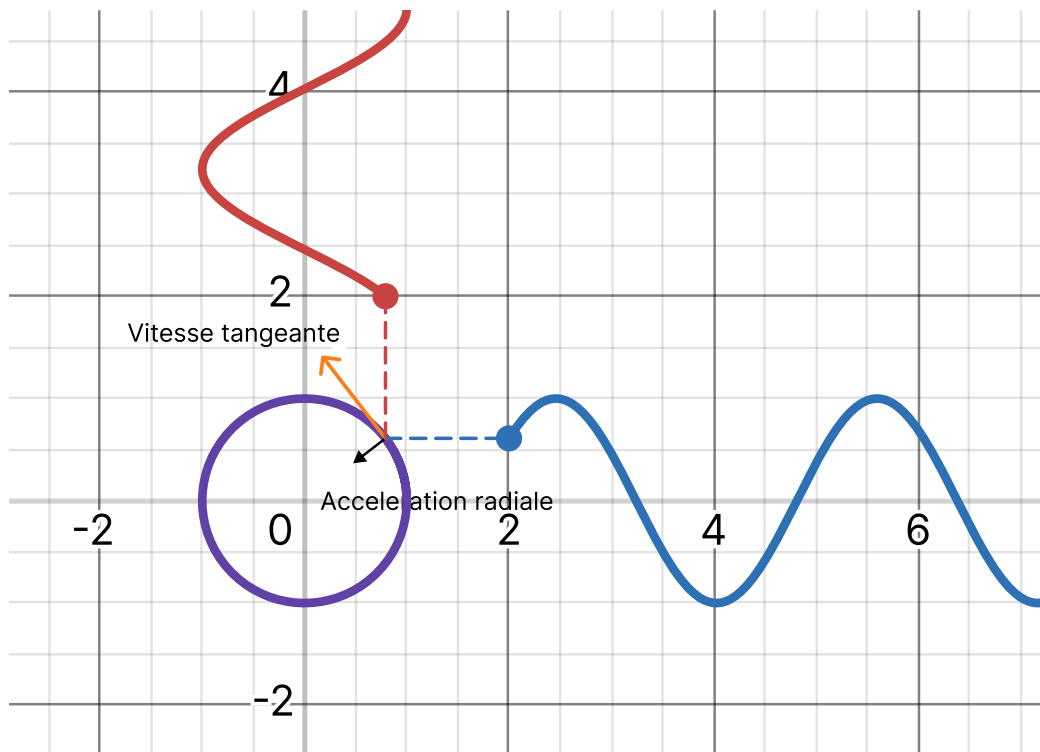


Fig. 7. – Le mouvement circulaire uniforme

3. Mouvement parabolique (projectile) :

- Position : $\vec{r}(t) = \begin{pmatrix} v_{0x} t \\ y_0 + v_{0y} t - \frac{1}{2} g t^2 \end{pmatrix}$
- Vitesse : $\vec{v}(t) = \begin{pmatrix} v_{0x} \\ v_{0y} - g t \end{pmatrix}$
- Accélération : $\vec{a}(t) = \begin{pmatrix} 0 \\ -g \end{pmatrix}$

L'accélération est **constante** et dirigée vers le bas. Avec $g = 9.81 \text{ m/s}^2$, $\vec{a} = \begin{pmatrix} 0 \\ -9.81 \end{pmatrix} \text{ m/s}^2$. Seule la composante verticale de la vitesse change.

Session 3 : Les lois de Newton et la méthode Euler

Objectifs de la Session :

- Réviser les sessions précédentes (la cinématique).
- Introduire les lois de Newton.
- Relier les précédentes sessions (la cinématique) aux lois de Newton.
- Introduire la méthode Euler pour résoudre les équations de la cinématique.
- Setup de git, vs code, live preview et Tinymist, pour avoir un accès en ligne et en local à l'ensemble des documents.
- Réaliser un premier travail pratique: Observer une de résolution d'équations cinématiques avec la méthode Euler dans un exemple simple.

Les 3 Lois de Newton

🔗 Physique du Jeu : Les 3 Lois de Newton

Ou « Comment programmer l'univers »

Isaac Newton n'a jamais codé en JavaScript, mais sans lui, aucun jeu vidéo moderne n'existerait. Ses trois lois sont les règles fondamentales du moteur physique que nous allons construire.

Les lois de Newton sont des lois mathématiques qui décrivent comment les objets se déplacent et se comportent sous l'effet des forces. Elles nous permettent de donner un comportement réaliste à nos objets dans le jeu.

En physique, on crée des modèles, en se basant sur nos observations, puis on confronte ces modèles à la réalité, pour valider ou invalider ces modèles. À aucun moment les physiciens prétendent que leurs modèles sont exactes, mais une loi est utile à partir du moment où aucune expérience ne peut prouver qu'elle est fausse.

1. La Première Loi : L'Inertie

Le principe du statu quo

La Loi

Un objet au repos reste au repos, et un objet en mouvement continue en ligne droite à vitesse constante, **sauf** si une force extérieure agit sur lui.

Ce que ça veut dire : Les objets ont une résistance naturelle au changement. Ils veulent continuer à faire ce qu'ils font déjà. Pour changer la vitesse d'un objet (accélérer, freiner, tourner), il **faut** appliquer une force.

Exemple « Vie de tous les jours » : Vous êtes debout dans le bus. Le bus freine brusquement. Votre corps continue d'avancer et vous manquez de tomber. C'est l'inertie : votre corps voulait garder sa vitesse.

Exemple « Jeu Vidéo » :

- **Space Invaders / Mario (Sol) :** Quand vous lâchez la manette, le personnage s'arrête instantanément. *C'est physiquement faux !* (C'est comme s'il y avait des frottements infinis).

- **Asteroids / Dead Space (Espace)** : Vous donnez une impulsion au vaisseau. Même si vous lâchez les commandes, le vaisseau continue d'avancer pour toujours dans la même direction. C'est ça, la vraie inertie (pas de frottements dans l'espace).
- **Niveaux de glace** : Pourquoi Mario glisse ? Parce que les frottements (la force extérieure qui freina) sont réduits. L'inertie reprend le dessus.

2. La Deuxième Loi : La Dynamique

L'équation du Moteur Physique

C'est la loi la plus importante pour nous, car c'est celle qu'on code littéralement dans `animate()`.

! La Formule

$\vec{F} = m \cdot \vec{a}$ Dans notre cas, on cherche l'accélération :

$$\vec{a} = \frac{\vec{F}}{m}$$

Ce que ça veut dire : L'accélération ($\vec{a}$) dépend de deux choses :

1. Une force : ($\vec{F}$).
2. La lourdeur intrinsèque de l'objet (m).

Exemple « Vie de tous les jours » : Imaginez un caddie de supermarché.

- **Vide (m petit)** : Une petite force le fait partir à toute vitesse.
- **Plein d'eau (m grand)** : La même force le fait à peine bouger.

Exemple « Jeu Vidéo » (Balancing) : Imaginez un jeu de tir (FPS) avec deux classes de personnages :

- **Le Scout (Masse = 50kg)** : Si le joueur appuie sur « Avancer » (Force = 500N), il accélère à 10m/s^2 . Il est agile.
- **Le Tank (Masse = 200kg)** : Avec la même touche « Avancer » (Force = 500N), il accélère seulement à 2.5m/s^2 . Il est lent et lourd.

Dans Three.js : C'est ici qu'on calcule `acceleration`. Si on veut simuler du vent, de la gravité ou des ressorts, on additionne toutes les forces, on divise par la masse, et on obtient l'accélération pour mettre à jour la vitesse en utilisant la méthode d'Euler. Et de la vitesse, on modifie la position, toujours en utilisant la méthode d'Euler une deuxième fois.

3. La Troisième Loi : Action-Réaction

Les forces sont des paires

La Loi

Pour toute action (force), il y a une réaction égale et opposée.

$$\vec{F}_{A \rightarrow B} = -\vec{F}_{B \rightarrow A}$$

Ce que ça veut dire : On ne peut pas toucher sans être touché. Si vous poussez un mur, le mur vous pousse en retour. Les forces vont toujours par paires.

Exemple « Vie de tous les jours » :

- **Le Skateboard** : Pour avancer, vous poussez le sol vers l'**arrière** avec votre pied. En réaction, le sol pousse votre pied (et vous) vers l'**avant**.
- **Le Ballon de baudruche** : L'air est éjecté vers l'arrière, le ballon part vers l'avant.

Exemple « Jeu Vidéo » :

- **Recul des armes (Recoil)** : Dans *Call of Duty*, quand vous tirez une balle vers l'avant (Action), votre arme et votre caméra remontent vers l'arrière (Réaction).
- **Rocket Jump (Quake / TF2)** : Vous tirez une roquette sur le sol. L'explosion exerce une force vers le bas sur le sol. En réaction, le sol (et l'explosion) exerce une force vers le haut sur le joueur, le propulsant dans les airs.
- **Collisions (Billard)** : Quand la boule blanche tape une boule rouge, la blanche s'arrête (ou recule) car la rouge lui a « rendu » le choc.

Résumé pour le développeur

Loi	Concept	Implémentation Code
1. Inertie	Les objets gardent leur vitesse.	Ne remettez pas velocity à 0 à chaque frame ! Laissez-la vivre entre les frames.
2. $F = ma$	Force → Accélération.	<code>acc = forces.divideScalar(mass)</code>
3. Action-Réaction	Les interactions sont doubles.	Collision : Si A repousse B, alors B doit repousser A avec la même force.

Tableau 1. – Aide-mémoire des lois de Newton

Annexe : Pourquoi des objets de masses différentes tombent-ils avec la même vitesse ?

La « force de pesanteur » est la force d'attraction gravitationnelle exercée par un astre (comme la Terre) sur un corps, et qui est aussi appelée « poids ». Elle est calculée par la formule :

$$\vec{P}_g = m \cdot \vec{g}$$

La deuxième loi de Newton nous dit que l'accélération est proportionnelle à la force :

$$\vec{a} = \frac{\vec{F}}{m} = \frac{m \cdot \vec{g}}{m} = \vec{g}$$

Donc, l'accélération est la même pour tous les objets, même si la force qui s'applique à eux est proportionnelle à leur masse.

Intégration Numérique & Série de Taylor

🔗 Comment l'ordinateur prédit le futur

Dans nos simulations (Three.js), le temps n'est pas continu. L'ordinateur calcule le monde image par image (environ 60 fois par seconde). Nous devons donc transformer les équations physiques continues (dérivées) en instructions discrètes (additions).

1. La Méthode d'Euler

La méthode d'Euler est l'approche la plus intuitive : elle suppose que la vitesse est constante entre deux images.

L'Algorithme (La boucle de jeu)

Pour chaque pas de temps Δt (ex: 0.016s) :

1. **Calculer l'accélération** (Forces / Masse) :

$$\vec{a}_n = \frac{\vec{F}}{m}$$

2. **Mettre à jour la vitesse** (Vitesse actuelle + Changement) :

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$$

3. **Mettre à jour la position** (Position actuelle + Déplacement) :

$$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \cdot \Delta t$$

2. D'où ça vient ? (La Dérivée Discrète)

Mathématiquement, la vitesse est la dérivée de la position. Si on enlève la limite $\lim_{\Delta t \rightarrow 0}$, on obtient une approximation :

$$\vec{v}(t) = \frac{d\vec{r}}{dt} \approx \frac{\vec{r}(t + \Delta t) - \vec{r}(t)}{\Delta t}$$

En isolant le terme futur $\vec{r}(t + \Delta t)$, on retombe sur la formule d'Euler :

$$\underbrace{\vec{r}(t + \Delta t)}_{\text{Futur}} \approx \underbrace{\vec{r}(t)}_{\text{Présent}} + \underbrace{\vec{v}(t) \cdot \Delta t}_{\text{Pas}}$$

3. La Série de Taylor (Le Moteur Mathématique)

La méthode d'Euler n'est en fait que la « pointe de l'iceberg » d'un concept plus puissant : la **Série de Taylor**. Elle stipule que n'importe quelle fonction (trajectoire) peut être reconstruite en additionnant ses dérivées successives.

L'intuition du conducteur aveugle : Si vous fermez les yeux au volant, comment deviner où vous serez dans 1 seconde (Δt) ?

- **Ordre 0 (Position)** : « Je ne bouge pas. » $\rightarrow \vec{r}(t)$
- **Ordre 1 (Vitesse)** : « J'avance tout droit. » $\rightarrow \vec{v}(t)\Delta t$
- **Ordre 2 (Accélération)** : « J'accélère, donc je courbe. » $\rightarrow \frac{1}{2}\vec{a}(t)\Delta t^2$
- **Ordre 3 (A-coup)** : « J'appuie de plus en plus fort... » $\rightarrow \dots$

La Formule Complète :

$$\vec{r}(t + \Delta t) = \vec{r}(t) + \vec{v}(t)\Delta t + \frac{1}{2}\vec{a}(t)\Delta t^2 + \frac{1}{6}\vec{j}(t)\Delta t^3 + ...$$

Pourquoi Euler est une approximation ?

La méthode d'Euler **coupe** la série de Taylor après le terme de vitesse.

$$\begin{aligned} \vec{r}_{n+1} &= \vec{r}_n + \vec{v}_n \Delta t \\ &+ \underbrace{O(\Delta t^2)}_{\text{Termes ignorés (Erreur)}} \end{aligned}$$

C'est pour cela qu'on dit qu'Euler est une méthode du **Premier Ordre**. L'erreur commise à chaque pas est proportionnelle au carré du pas de temps (Δt^2).

4. Conséquences Pratiques

Puisque Euler ignore la courbure de la trajectoire (l'accélération) à l'intérieur d'un pas de temps, il a tendance à « déraiper » vers l'extérieur des virages ou à gagner de l'énergie (spirale vers l'extérieur) dans des systèmes orbitaux.

Euler (Ordre 1)	Intégration exacte (Ordre 2)
$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t$ <i>Prend la pente au début et trace une ligne droite.</i>	$\vec{r}_{n+1} = \vec{r}_n + \vec{v}_n \Delta t + \frac{1}{2}\vec{a}\Delta t^2$ <i>Prend en compte la courbure (accélération).</i>
Facile à coder. Rapide.	Exact pour la gravité constante (projectiles).

Tableau 2. – Comparaison pour un pas de temps Δt

Conclusion pour le projet : Pour des projectiles simples soumis à une gravité constante, nous pourrions utiliser la formule exacte (Ordre 2). Cependant, Euler reste très utile car il fonctionne même quand l'accélération change tout le temps (ex: vent, ressorts, collisions), là où la formule exacte devient trop complexe.

Session 4 : Les Forces de Frottement

💡 Introduction

Dans le vide spatial (et dans un moteur physique naïf), un objet lancé garde sa vitesse pour l'éternité (1ère loi de Newton).

Sur Terre, tout finit par s'arrêter. Pourquoi ? À cause des interactions avec l'environnement. Ces interactions sont modélisées par des Forces de Frottement.

Il existe deux grandes familles à maîtriser pour le jeu vidéo :

1. Le Frottement Sec (Solide contre Solide).
2. Le Frottement Visqueux (Solide dans un Fluide).

1. Le Frottement Sec (Modèle de Coulomb)

C'est le frottement de contact : un pneu sur la route, une caisse sur le sol, ou une boule de billard qui glisse. Tous les frottements qui « accrochent ».

Il ne dépend pas de la vitesse (tant que ça bouge), ni de la surface (l'aire) de contact (contrairement à l'intuition).

Il dépend de deux choses :

1. **La rugosité des matériaux** (Coefficient μ).
2. **Le poids apparent** (Force Normale N).

$$\|\vec{f}\| = \mu \cdot \|\vec{N}\|$$

Direction : Toujours opposée à la vitesse (ou à la force qui tente de bouger l'objet). Il compense la force qui tente de bouger l'objet, et le garde immobile. Il va s'adapter exactement pour annuler votre poussée, jusqu'à un seuil critique.

La Force Normale (N)

C'est la force avec laquelle les deux surfaces sont pressées l'une contre l'autre.

- Sur un sol plat horizontal : $N = mg$ (Le Poids).
- Sur une pente : $N = mg \cos(\theta)$ (La composante perpendiculaire au sol).
- Dans un virage relevé (F1) : N augmente à cause de la force centrifuge.

-Bien assis sur son siège : $N = mg \cos(\alpha)$ (la composante perpendiculaire au sol).

💬 Distinction : Statique vs Cinétique

Avez-vous remarqué qu'il est plus dur de **commencer** à pousser une armoire lourde que de la **maintenir** en mouvement ? On pousse de plus en plus fort et tout d'un coup on passe le seuil critique, et l'armoire commence à bouger beaucoup plus facilement.

A. Frottement Statique (μ_s)

- L'objet est immobile ($v = 0$).

- La force est « intelligente » : elle s'adapte exactement pour annuler votre poussée, jusqu'à un seuil critique.
- **Seuil de rupture** : $F_{\max} = \mu_s N$.

B. Frottement Cinétique (μ_k)

- L'objet glisse ($v > 0$).
- La force est **constante**, peu importe la vitesse.
- $F_k = \mu_k N$.

En général : $\mu_s > \mu_k$. (C'est pour ça que l'armoire « décolle » d'un coup).

2. Le Frottement Visqueux (Fluide)

C'est la résistance de l'air (Drag) ou de l'eau. Contrairement au frottement sec, celui-ci dépend énormément de la vitesse, ainsi que la forme de l'objet.

Cas 1 : Vitesse faible (Loi de Stokes) Pour les particules dans le miel ou la poussière.

$$\vec{f} = -k \cdot \vec{v}$$

(Proportionnel à la vitesse).

Cas 2 : Vitesse élevée (Loi quadratique) Pour les voitures, les avions, les balles.

$$\vec{f} = -C \cdot \|v\|^2 \cdot \hat{v}$$

(Proportionnel au **carré** de la vitesse).

Conséquence : La Vitesse Terminale Si un objet tombe, la gravité (mg) est constante, mais le frottement fluide augmente. À un moment, les deux s'annulent : l'objet ne peut plus accélérer.

Exemple. Comparaison pour le Gameplay :

- **Frottement Sec (Billard, Glisse)** : L'objet ralentit linéairement (v diminue de manière constante). Il s'arrête net.
- **Frottement Fluide (Amortissement)** : L'objet ralentit de moins en moins vite. Il ne s'arrête théoriquement jamais complètement (approche asymptotique de 0).

TP : Le Laboratoire de Chocs 1D

🔗 Objectifs du TP

Nous allons implémenter la réponse physique d'une collision entre deux boules de masses différentes.

Pour simplifier le problème et isoler les mathématiques de l'impulsion, nous travaillons dans un « Laboratoire 1D » :

- Pas de gravité.
- Mouvement uniquement sur l'axe X.
- Pas de friction pour l'instant.

1. Analyse de la Scène

Le code de démarrage (`Template.js`) contient :

- Une **Boule Rouge (A)** : Lourde ($m = 5$), lancée vers la droite.
- Une **Boule Verte (B)** : Légère ($m = 1$), immobile (actuellement désactivée).
- Une interface **GUI** : Pour modifier les masses, la vitesse et le temps en direct.

État actuel : La boule rouge traverse le vide et rebondit bêtement sur les murs (inversion simple de vitesse).

2. Rappel Théorique : L'Impulsion

Lors d'un choc, nous devons calculer une impulsion j (scalaire) et l'appliquer aux deux objets.

Formule de l'Impulsion (avec Masse Réduite) :

$$j = -(1 + e) \cdot v_{\text{rel}} \cdot \left(\frac{m_A m_B}{m_A + m_B} \right)$$

- e : Coefficient de restitution (1 = Élastique, 0 = Mou).
- v_{rel} : Vitesse relative de rapprochement

$$(\vec{v}_A - \vec{v}_B) \cdot \vec{n}$$

.

Application (Action-Réaction) :

$$\vec{v}'_A = \vec{v}_A + \left(\frac{j}{m_A} \right) \vec{n}$$

$$\vec{v}'_B = \vec{v}_B - \left(\frac{j}{m_B} \right) \vec{n}$$

3. Vos Missions (Code)

Étape A : Activer la Boule B Dans le fichier `init()`, décommentez les lignes créant la deuxième boule. Relancez : les boules devraient se traverser comme des fantômes.

Étape B : Implémenter le « Solver » (Fonction `resolveBallCollision`) Repérez la zone `TODO ÉTUDIANT`. Vous devez traduire la physique en JavaScript (Three.js).

1. Calculez la Normale $\vec{n}$ (Vecteur unitaire de B vers A).
2. Calculez la Vitesse Relative v_{rel} (Produit scalaire).
3. **Sécurité** : Si $v_{\text{rel}} > 0$ (elles s'éloignent), on arrête (`return`).
4. Calculez la Masse Réduite et l'Impulsion j .
5. Appliquez le changement aux vitesses `velA` et `velB`.

Indice : Utilisez `addScaledVector(vector, scale)`.

4. Expérimentations (Observations)

Une fois votre code fonctionnel, utilisez la GUI pour tester ces scénarios physiques classiques.

Scénario 1 : Le Pendule de Newton

- Réglez $m_A = 5$ et $m_B = 5$ (Masses égales).
- Restitution $e = 1$.
- **Observation** : La boule rouge doit s'arrêter net. La verte doit partir avec toute la vitesse.

Scénario 2 : Le Mur

- Réglez $m_A = 1$ (Légère) et $m_B = 100$ (Très lourde).
- **Observation** : La rouge rebondit en arrière. La verte bouge à peine.

Scénario 3 : Le Bulldozer (Effet Catapulte)

- Réglez $m_A = 100$ (Très lourde) et $m_B = 1$ (Légère).
- **Observation** : La rouge continue presque sans ralentir. À quelle vitesse part la verte ? (Indice : $2 \times v_A$).

Scénario 4 : La Pâte à modeler

- Mettez la restitution $e = 0$.
- **Observation** : Les boules doivent se coller et avancer ensemble à une vitesse moyenne.

💡 Astuce Debug

Si vos boules disparaissent ou accélèrent à l'infini :

1. Vérifiez le signe dans l'application de l'impulsion ($-j$ pour B).
2. Vérifiez la condition `if (vRel > 0) return;`.
3. Utilisez le slider « Time Scale » pour mettre le temps au ralenti (0.1) et voir exactement ce qui se passe au moment de l'impact.

Annexe : L'Impulsion de Collision

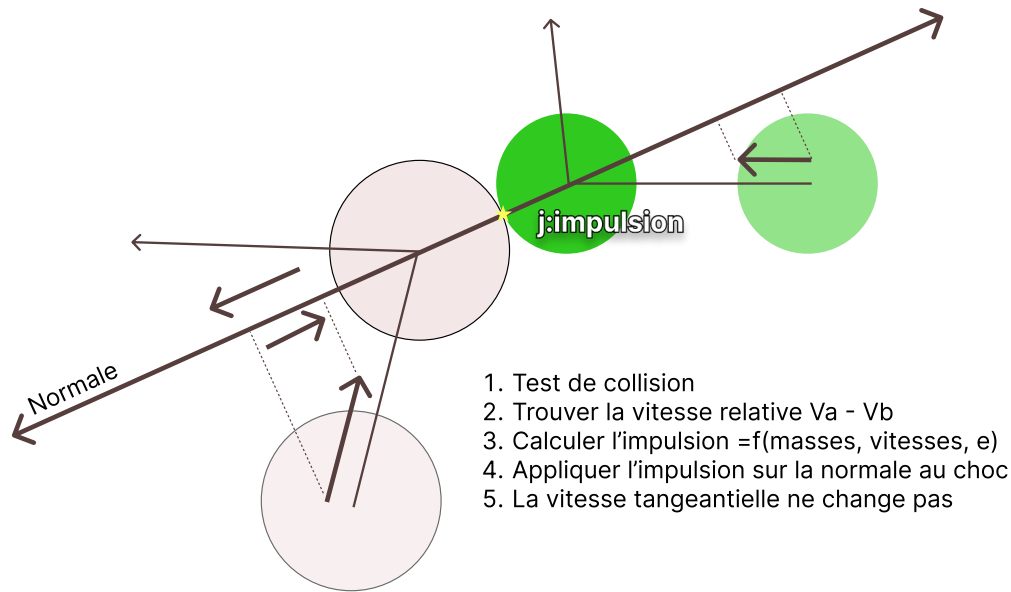


Fig. 8. – Choc entre deux objets

Partie 1 : D'où vient l'Impulsion ? (Le Lien Newtonien)

La conservation de la quantité de mouvement est une loi fondamentale de la physique, et elle nous permet de calculer facilement les changements de vitesse de 2 objets en collision.

1. Retour à la définition de la Force L'accélération $\vec{a}$ est la dérivée de la vitesse.

$$\vec{F} = m\vec{a} = m \frac{d\vec{v}}{dt}$$

Comme la masse est constante, on peut la rentrer dans la dérivée. On reconnait alors la Quantité de Mouvement ($\vec{p} = m\vec{v}$).

$$\vec{F} = \frac{d(m\vec{v})}{dt} = \frac{d\vec{p}}{dt}$$

2. L'Intégration (Somme des forces) Pour connaître l'effet total d'un choc entre un instant t_1 et t_2 , on multiplie par dt et on intègre (on somme) les deux côtés de l'équation :

$$\int_{t_1}^{t_2} \vec{F} dt = \int_{p_1}^{p_2} d\vec{p}$$

3. Le Résultat : Le Théorème de l'Impulsion

- Le terme de droite devient simplement le changement de quantité de mouvement : $\Delta\vec{p}$.
- Le terme de gauche est la définition exacte de l'Impulsion $\vec{J}$.

$$\vec{J} = \Delta\vec{p}$$

Interprétation Géométrique : L'impulsion est l'**Aire sous la courbe** de la Force en fonction du temps. Peu importe que la force soit petite et longue, ou géante et courte : si l'aire est la même, le changement de vitesse à l'instant t_2 est le même.

Dans une simulation « Narrow Phase », on considère que la collision est instantanée ($\Delta t \approx 0$).

Si le temps est nul, pour avoir une aire non-nulle, la Force devrait être infinie. L'ordinateur ne peut pas calculer l'infini.

L'Astuce : On saute l'étape de l'intégration temporelle. On utilise directement le résultat J . Cela nous permet de « téléporter » la vitesse sans passer par l'accélération.

$$\Delta v = \frac{J}{m}$$

Partie 2 : La Règle du Rebond (La Loi)

Nous avons l'outil (J), mais quelle valeur doit-il avoir ? C'est ici qu'intervient la **Loi expérimentale**.

Loi de Restitution : « La vitesse à laquelle deux objets s'éloignent après un choc est proportionnelle à la vitesse à laquelle ils se rapprochaient avant le choc. »

$$v_{\text{rel}}(\text{après}) = -e \cdot v_{\text{rel}}(\text{avant})$$

- $v_{\text{rel}} = v_A - v_B$ (Vitesse relative le long de la normale).
- e : Coefficient de restitution (entre 0 et 1).
- Le signe moins (−) indique que les objets repartent dans l'autre sens.

Zoom sur la Vitesse Relative

Avant de calculer, définissons les termes. Soient $\vec{v}_A$ et $\vec{v}_B$ les vecteurs vitesse des deux objets dans le monde.

1. Le Vecteur Vitesse Relative ($\Delta \vec{v}$) C'est la vitesse de A vue depuis B.

$$\vec{v}_{\text{diff}} = \vec{v}_A - \vec{v}_B$$

2. La Normale de Collision ($\vec{n}$) C'est le vecteur unitaire qui relie les centres (la direction du choc).

3. La Vitesse Relative Scalaire (v_{rel}) C'est celle qu'on utilise dans la formule de l'impulsion ! On ne s'intéresse qu'à la vitesse **sur l'axe du choc**. On fait donc une projection (Produit Scalaire) :

$$v_{\text{rel}} = (\vec{v}_A - \vec{v}_B) \cdot \vec{n}$$

Interprétation du signe :

- Si $v_{\text{rel}} < 0$: Les objets se rapprochent (Collision imminente).
- Si $v_{\text{rel}} > 0$: Les objets s'éloignent (Ils se sont déjà tapés ou se fuient).
- Si $v_{\text{rel}} = 0$: Ils glissent l'un à côté de l'autre sans s'écraser.

Application de l'Impulsion

Cherchons l'intensité de l'impulsion j (scalaire) à appliquer le long de la normale pour satisfaire la loi du rebond.

On applique j sur la boule A et $-j$ sur la boule B (3ème loi de Newton). Les nouvelles vitesses (v') sont :

$$v'_A = v_A + \frac{j}{m_A} \quad \text{et} \quad v'_B = v_B - \frac{j}{m_B}$$

En passant par les impulsions, on peut effectuer un changement de vitesse instantané, sans passer par l'accélération.

Calcul de la valeur de l'impulsion

Voici la formule de base pour calculer l'impulsion :

$$j = \frac{-(1+e)v_{\text{rel}}}{\frac{1}{m_A} + \frac{1}{m_B}}$$

Le terme au dénominateur $\left(\frac{1}{m_A} + \frac{1}{m_B}\right)$ est peu intuitif. Transformons-le. Mise au même dénominateur :

$$\frac{1}{m_A} + \frac{1}{m_B} = \frac{m_B}{m_A m_B} + \frac{m_A}{m_A m_B} = \frac{m_A + m_B}{m_A m_B}$$

Maintenant, remplaçons cette fraction dans la formule de j . Diviser par une fraction, c'est multiplier par son inverse :

$$j = -(1+e)v_{\text{rel}} \cdot \frac{1}{\frac{m_A + m_B}{m_A m_B}}$$

Ce qui nous donne la **Formule Finale** très élégante :

$$j = \underbrace{-(1+e)v_{\text{rel}}}_{\text{Vitesse à changer}} \cdot \underbrace{\left(\frac{m_A m_B}{m_A + m_B}\right)}_{\text{Masse Équivalente}}$$

💡 Interprétation Physique

Cette forme

$$\frac{m_A m_B}{m_A + m_B}$$

est appelée la **Masse Réduite** du système.

Elle montre que lors d'une collision, le système se comporte comme une masse unique équivalente.

- Si une masse est minuscule ($m_A \ll m_B$), la masse équivalente devient $\approx m_A$. **Conclusion** : C'est le plus léger qui dicte la dynamique du rebond.

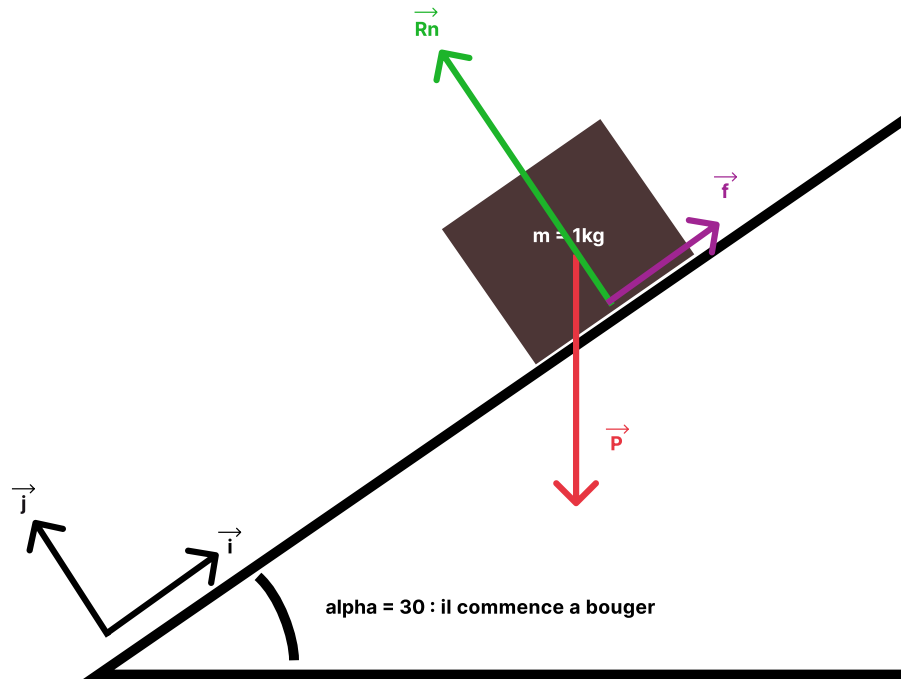
Exercice : Frottement sur Plan Incliné

Énoncé du problème. Un cube de masse $m = 1$ kg est posé sur un plan incliné.

On augmente progressivement l'inclinaison du plan. Lorsque l'angle atteint $\theta = 30^\circ$, le cube commence tout juste à glisser.

Question : Quelle est la valeur limite de la force de frottement statique ($f_{s, \max}$) à cet instant précis ?

(On prendra $g = 9.81 \text{ m/s}^2$)

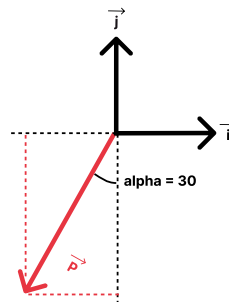


Solution Détaillée

1. Bilan des Forces Le cube est soumis à 3 forces :

- Le Poids ($\vec{P}$) : Vertical, vers le bas.
- La Normale ($\vec{N}$) : Perpendiculaire au plan.
- Le Frottement Statique ($\vec{f}_s$) : Parallèle au plan, vers le haut (il retient le cube).

2. Décomposition du Poids Pour simplifier le calcul, on place notre repère aligné avec le plan incliné.



Il faut décomposer le vecteur Poids en deux composantes :

- $P_y = mg \cos(\theta)$ (Composante qui plaque le cube contre le sol).
- $P_x = mg \sin(\theta)$ (Composante qui tire le cube vers le bas de la pente).

3. Condition d'équilibre limite Juste avant que le cube ne bouge, les forces s'annulent parfaitement. Sur l'axe X (le long de la pente), la force qui tire vers le bas (P_x) est exactement compensée par la force de frottement maximale ($f_{s, \max}$).

$$\sum F_x = 0 \text{ implique } P_x - f_{s, \max} = 0$$

Donc :

$$f_{s, \max} = P_x = mg \sin(\theta)$$

4. Application Numérique

$$m = 1 \text{ kg}$$

$$g = 9.81 \text{ m/s}^2$$

$$\theta = 30^\circ$$

$$f_{s, \max} = 1 \cdot 9.81 \cdot \sin(30^\circ)$$

$$f_{s, \max} = 9.81 \cdot 0.5$$

Réponse :

$$f_{s, \max} \approx 4.905 \text{ Newtons}$$

Pour aller plus loin : Le Coefficient de Frottement

On sait que la friction statique maximale dépend de la force Normale :

$$f_{s, \max} = \mu_s \cdot N$$

Or, la Normale compense la composante du poids perpendiculaire (P_y) :

$$N = mg \cos(\theta)$$

En combinant les deux équations :

$$mg \sin(\theta) = \mu_s \cdot (mg \cos(\theta))$$

Les masses s'annulent ! On isole μ_s :

$$\mu_s = \frac{\sin(\theta)}{\cos(\theta)} = \tan(\theta)$$

Résultat magique : Le coefficient de frottement est simplement la tangente de l'angle limite.

$$\mu_s = \tan(30^\circ) \approx 0.577$$

Session 5 : Cahier d'Exercices - Vecteurs & Physique

Thème 1 : Vecteurs et Produits Scalaires

Ces exercices visent à maîtriser les outils mathématiques de base pour la détection et l'IA.

Exercice 1 : Le Garde et le Voleur. **Contexte :** Dans un jeu d'infiltration, un garde regarde dans une direction donnée par le vecteur directeur $\vec{D}$. Un voleur se trouve à une position P dans la pièce. Le garde se trouve à la position G . On considère que le garde a un champ de vision de 180° (il voit tout ce qui est devant lui). **Données :**

- Position du Garde : $G(10, 10)$
- Direction du regard : $\vec{D} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ (Regarde vers l'Est)
- Position du Voleur : $P(5, 12)$

Question : En utilisant le produit scalaire, déterminer si le voleur est « Devant » ou « Derrière » le garde.

Exercice 2 : La Racing Line (Projections). **Contexte :** Pour aider une IA à conduire, on définit la trajectoire idéale comme une ligne droite entre deux points A et B . La voiture se trouve actuellement au point V . On souhaite connaître la distance latérale entre la voiture et cette trajectoire idéale (l'écart de conduite). **Données :**

- Départ de la ligne : $A(0, 0)$
- Fin de la ligne : $B(100, 0)$
- Position Voiture : $V(50, 5)$

Question :

1. Calculer le vecteur $\overrightarrow{AV}$.
2. Projeter ce vecteur sur la direction de la piste $\overrightarrow{AB}$.
3. En déduire la distance de la voiture à la ligne.

Exercice 3 : Le Radar de Vitesse (Vitesse Relative). **Contexte :** Un radar mesure la vitesse de rapprochement sur sa ligne de visée. Pour savoir ce que le radar affiche, il faut projeter les vitesses des véhicules sur l'axe qui les relie. **Données (Positions et Vitesses) :**

- Police (A) :
 - Position : $P_A = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$
 - Vitesse : $\vec{v}_A = \begin{pmatrix} 0 \\ 100 \end{pmatrix}$ (Roule vers le Nord à 100 km/h)
- Cible (B) :
 - Position : $P_B = \begin{pmatrix} 0 \\ 500 \end{pmatrix}$ (Est située 500m au Nord de la police)
 - Vitesse : $\vec{v}_B = \begin{pmatrix} 85 \\ 85 \end{pmatrix}$ (Roule vers le Nord-Est à ≈ 120 km/h)

Questions :

1. Calculer le vecteur direction de visée $\vec{D} = P_B - P_A$.
2. En déduire la normale de visée $\vec{n}$ (le vecteur unitaire).
3. Projeter les vitesses $\vec{v}_A$ et $\vec{v}_B$ sur $\vec{n}$ (Produit Scalaire).
4. La police est-elle en train de rattraper la cible ? (Est-ce que v_A projeté $>$ v_B projeté ?)

Thème 2 : Forces, Gravité et Frottements

Application des lois de Newton pour simuler des mouvements réalistes.

Exercice 4 : Le Drag Race (Code). **Contexte :** Une voiture accélère en ligne droite.

- Force Moteur : $F_{\text{moteur}} = 5000\text{N}$ (Constante).
- Friction de l'air : $F_{\text{air}} = -k \cdot v^2$ (Opposée au mouvement).
- Masse : $m = 1000\text{ kg}$, Coefficient $k = 0.8$.

Objectif Code : Implémenter ce système dans Three.js. Afficher la vitesse en temps réel. Observer la « Vitesse Terminale » : le moment où la voiture n'accélère plus car la friction compense exactement le moteur.

Exercice 5 : Le Sniper (Balistique). **Contexte :** On tire un projectile soumis uniquement à la gravité $\vec{g} = \begin{pmatrix} 0 \\ -9.81 \end{pmatrix}$. **Données :**

- Vitesse initiale : $v_0 = 50\text{ m/s}$.
- Angle de tir : $\theta = 45^\circ$.
- Hauteur initiale : $y_0 = 0$.

Objectif Code : Trouver ce résultat dans une simulation avec la méthode d'Euler.

Exercice 6 : Le Cube sur la Pente (Code). **Contexte :** Reprise de l'exercice théorique sur le plan incliné. **Mise en place :** Créer un plan (sol) dans Three.js dont on peut modifier l'inclinaison (rotation X) via une interface (GUI). Poser un cube dessus. **Logique à coder :**

- Calculer la force de gravité parallèle à la pente : $P_x = mg \sin(\theta)$.
- Calculer la friction statique max : $f_{s, \max} = \mu_s mg \cos(\theta)$.
- **Condition :** Tant que $P_x < f_{s, \max}$, la vitesse reste à 0. Sinon, le cube accélère.

Test : Vérifier si le cube se met à glisser exactement à l'angle prévu (ex: 30° pour $\mu_s = 0.57$).

Exercice 7 : L'Ascenseur Spatial (Ressorts). **Contexte :** Une boule est suspendue dans le vide, attachée à un point fixe par un ressort invisible. **Loi de Hooke :** $\vec{F} = -k \cdot (L - L_0) \cdot \vec{n}$ (Où L est la longueur actuelle et L_0 la longueur au repos). **Objectif Code :** Simuler ce ressort. Ajouter une force d'amortissement (Damping) $F = -b \cdot v$ pour que la boule finisse par se stabiliser et ne rebondisse pas à l'infini.

Définition 1 (Thème 3 : Collisions et Impulsions)

Gestion des chocs et conservation de la quantité de mouvement.

Exercice 8 : Le Bumper Car (Code). **Contexte :** Arène fermée avec 5 boules de masses identiques.

Objectif Code : Mettre en place la boucle de détection « Narrow Phase » : vérifier chaque paire de boules (i, j) . Si $\text{distance} < \text{radius} * 2$, résoudre la collision élastique ($e = 1$). Observer le chaos et vérifier que les boules ne « fusionnent » pas (stabilité).

Exercice 9 : Le Matériau Mystère. **Contexte :** On lâche 3 boules de la même hauteur $H = 10m$. Elles tombent sur le sol (masse infinie).

- Boule A : Remonte à 10m.
- Boule B : Remonte à 5m.
- Boule C : Reste collée au sol.

Question : Déterminer le coefficient de restitution e pour chaque boule. Rappel : $v_{\text{après}} = -e \cdot v_{\text{avant}}$.

Exercice 10 : Le Rocket Jump. **Contexte :** Une explosion se produit à la position $E(0, 0, 0)$. Un joueur se trouve à la position $P(2, 0, 0)$. On veut propulser le joueur avec une impulsion radiale. **Règle :**

- Direction : Du centre de l'explosion vers le joueur.
- Intensité : $J = \frac{100}{\text{distance}^2}$.

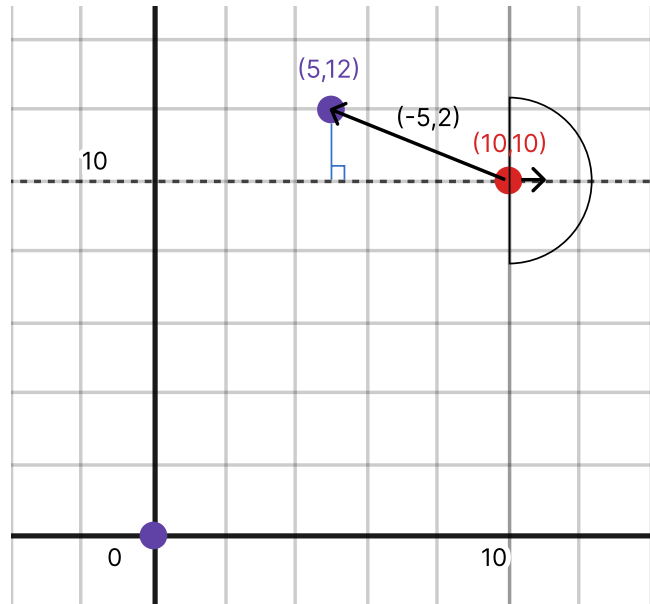
Question : Calculer le vecteur Impulsion $\vec{J}$ à appliquer au joueur.

Corrigé : Vecteurs & Physique

Thème 1 : Vecteurs et Produits Scalaires

Solutions détaillées.

Solution 1 : Le Garde et le Voleur



1. **Calcul du vecteur Garde $\rightarrow$ Voleur ($\vec{V}$) :**

$$\vec{V} = P - G = \begin{pmatrix} 5 - 10 \\ 12 - 10 \end{pmatrix} = \begin{pmatrix} -5 \\ 2 \end{pmatrix}$$

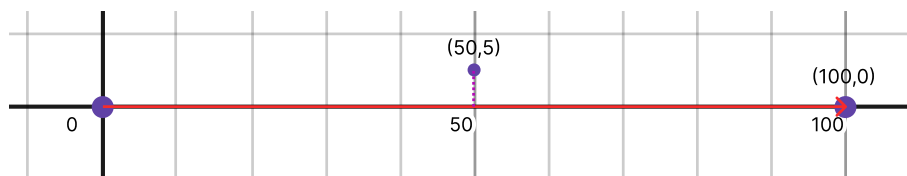
2. **Produit Scalaire (Test de vision) :** On projette la position du voleur sur la direction du regard.

$$\text{Dot} = \vec{V} \cdot \vec{D} = (-5 \times 1) + (2 \times 0) = -5$$

3. **Conclusion :** Le résultat est négatif ($-5 < 0$). L'angle entre le regard et le voleur est donc supérieur à 90° .

Réponse : Le voleur est Derrière le garde.

Solution 2 : La Racing Line



1. **Vecteur Voiture ($\vec{AV}$) :**

$$\overrightarrow{AV} = V - A = \begin{pmatrix} 50 \\ 5 \end{pmatrix}$$

2. **Direction de la piste ($\vec{u}$)** : Le vecteur $\overrightarrow{AB} = \begin{pmatrix} 100 \\ 0 \end{pmatrix}$. Son vecteur unitaire (normalisé) est $\vec{u} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$.

3. **Projection (Distance parcourue sur la piste)** :

$$\text{Proj} = \overrightarrow{AV} \cdot \vec{u} = (50 \times 1) + (5 \times 0) = 50$$

Le point projeté sur la ligne est donc $H(50, 0)$.

4. **Distance latérale (Écart)** : C'est la distance entre $V(50, 5)$ et $H(50, 0)$.

Réponse : La voiture est à 5 mètres de la ligne idéale.

Solution 3 : Le Radar (Vitesse Relative)

1. **Vecteur de visée $\vec{D}$** :

$$\vec{D} = P_B - P_A = \begin{pmatrix} 0 \\ 500 \end{pmatrix} - \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 500 \end{pmatrix}$$

2. **Normale de visée $\vec{n}$** : On normalise $\vec{D}$. Comme il est purement vertical :

$$\vec{n} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

3. **Projections des vitesses (Produit Scalaire)** :

- Police : $v_{A_{\text{proj}}} = \begin{pmatrix} 0 \\ 100 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 100 \text{ km/h}$
- Cible : $v_{B_{\text{proj}}} = \begin{pmatrix} 85 \\ 85 \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 85 \text{ km/h}$

4. **Conclusion** : La police fonce vers le Nord à 100. La cible s'éloigne vers le Nord à 85 seulement (le reste de sa vitesse part vers l'Est).

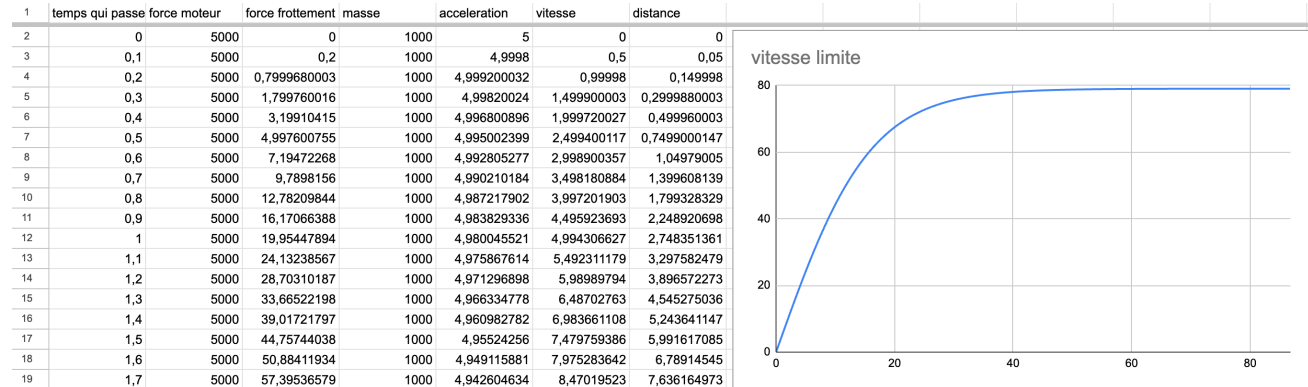
$$v_{A_{\text{proj}}} > v_{B_{\text{proj}}}$$

Réponse : Oui, la police rattrape la cible (à une vitesse relative de 15 km/h). Mais attention, cela n'est vrai qu'au moment du scan. Comme la voiture de la cible se déplace vers l'est, le vecteur de visée change constamment.

Thème 2 : Forces, Gravité et Frottements

Solutions et Logique Code.

Solution 4 : Le Drag Race



Physique : La « Vitesse Terminale » est atteinte quand l'accélération est nulle, donc quand les forces s'annulent.

$$\sum F = 0 \text{ implique } F_{\text{moteur}} + F_{\text{air}} = 0$$

$$5000 - kv^2 = 0$$

$$kv^2 = 5000$$

Calcul :

$$v = \sqrt{\frac{5000}{0.8}} = \sqrt{6250} \approx 79.05 \text{ m/s}$$

Soit environ 284 km/h.

Solution 5 : Le Sniper

1. Décomposition de la vitesse initiale ($t = 0$)

Avec $v_0 = 50 \text{ m/s}$ et $\theta = 45^\circ$:

$$v_{\{0x\}} = v_0 \cos(\theta) = 50 \times 0.707 \approx 35.35 \text{ m/s}$$

$$v_{\{0y\}} = v_0 \sin(\theta) = 50 \times 0.707 \approx 35.35 \text{ m/s}$$

2. Équations de position (au temps t)

On néglige les frottements. L'accélération ne joue que sur la hauteur (y) :

- Horizontal : $x(t) = v_{\{0x\}} \times t$
- Vertical : $y(t) = v_{\{0y\}} \times t - \frac{1}{2}gt^2$

3. Calcul du temps de vol ($t_{\{vol\}}$)

Le projectile touche le sol quand $y(t) = 0$ (pour $t > 0$) :

$$35.35t - 0.5 \times 9.81 \times t^2 = 0$$

$$t \times (35.35 - 4.905t) = 0$$

$$t_{\text{vol}} = \frac{35.35}{4.905} \approx 7.207 \text{ secondes}$$

4. Calcul de la portée (R)

C'est la distance parcourue horizontalement pendant le temps de vol :

$$R = x(t_{\text{vol}}) = 35.35 \text{ m/s} \times 7.207\text{s} \approx 254.8 \text{ mètres}$$

Note Code : Si votre simulation (Euler) donne une valeur éloignée (ex: 260m), c'est que votre pas de temps Δt est trop grand pour capter précisément l'instant t_{vol} .

Solution 6 : Le Cube (Logique Code)

Algorithme à implémenter dans updatePhysics :

```
// 1. Calcul des forces
const Px = mass * g * Math.sin(angle); // Force qui tire vers le bas
const Py = mass * g * Math.cos(angle); // Force qui plaque au sol

const frictionMax = mu * Py; // Résistance max du sol

// 2. Décision (Seuil)
if (Px > frictionMax) {
    // Glissement : La force nette est la différence
    const netForce = Px - (mu_cinetique * Py);
    acceleration = netForce / mass;
} else {
    // Statique : Pas de mouvement
    acceleration = 0;
    velocity = 0;
}
```

Session 6 : Broad & Narrow Phases

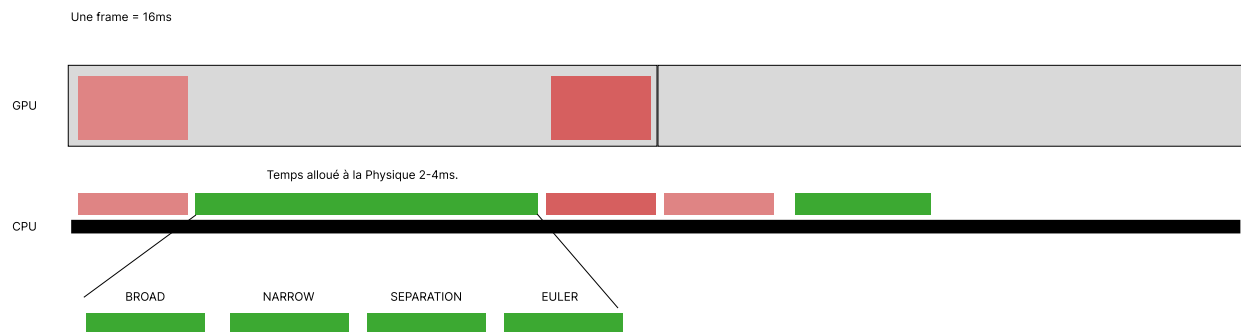
1. Le Contexte : Le Budget Temps (16ms)

Dans un jeu vidéo tournant à 60 FPS, vous avez 16.6 millisecondes pour **tout** calculer.

La physique n'est qu'une part du gâteau :

- Rendu (Graphics) : 8ms
- IA / Gameplay : 4ms
- Audio / Réseau : 2ms
- **Physique : 2ms**

Si votre détection de collision prend 10ms, le jeu saccade (« lag »). L'optimisation n'est pas un bonus, c'est une nécessité vitale.



2. Le Problème Mathématique : La Malédiction $O(N^2)$

Pourquoi la détection « naïve » échoue-t-elle ? Imaginons une matrice d'interaction où chaque objet (Ligne) teste chaque objet (Colonne).

Pour $N = 5$ objets (A, B, C, D, E) :

	A	B	C	D	E
A	X	Test	Test	Test	Test
B	Skip	X	Test	Test	Test
C	Skip	Skip	X	Test	Test
D	Skip	Skip	Skip	X	Test
E	Skip	Skip	Skip	Skip	X

- La Diagonale (X) : Test inutile (A vs A).
- Le triangle inférieur (Skip) : Redondant (si A touche B, B touche A).

Le Coût Réel :

$$C \approx \frac{N(N-1)}{2} \approx O(N^2)$$

- 10 objets → 45 tests (Négligeable)

- 1 000 objets $\rightarrow$ 499 500 tests (Lent)
- 10 000 objets $\rightarrow$ 50 000 000 tests (Crash CPU)

Solution : Il faut réduire N avant de faire les tests coûteux. C'est le rôle de la Broad Phase.

La Broad Phase

3. La Broad Phase : Structures de Partitionnement

Le but est de répondre rapidement à la question : « **Qui sont mes voisins ?** ». On remplace les objets complexes par des AABB (Axis Aligned Bounding Box).

A. Grille Uniforme (Spatial Hashing)

- Concept : On découpe le monde en cases de taille fixe. On ne teste l'objet A que contre les objets dans la même case (ou les cases adjacentes).
- Avantage : Accès $O(1)$, très rapide, facile à coder.
- Inconvénient : Mauvais si le monde est infini ou si les objets ont des tailles très variées.

B. Arbres Spatiaux (Quadtree / Octree)

- Concept : Découpage récursif. Si une case contient trop d'objets, on la coupe en 4 (2D) ou 8 (3D).
- Avantage : S'adapte à la densité (beaucoup de cases là où il y a des objets, peu ailleurs).
- Inconvénient : Coûteux à mettre à jour si les objets bougent beaucoup.

C. Sweep and Prune (SAP)

- Concept : On projette les débuts/fins des AABB sur les axes X, Y, Z et on trie la liste. Si les intervalles sur X ne se chevauchent pas, inutile de tester Y et Z.
- Secret : Exploite la **Cohérence Temporelle** (les objets ne se téléportent pas, la liste reste presque triée d'une frame à l'autre).

La Narrow Phase

4. La Narrow Phase : Les Méthodes de Résolution

On a filtré 99% des cas. Il reste les paires « proches ». Comment savoir si elles se touchent vraiment ? Deux écoles dominent :

Méthode 1 : SAT (Separating Axis Theorem)

- Philosophie : « L'ombre ». Si je peux trouver une lumière qui projette des ombres séparées des deux objets, alors ils ne se touchent pas.
- Fonctionnement : On projette les formes sur une série d'axes (Normales des faces, Produits vectoriels des arêtes). Si **une seule** projection montre un écart, il n'y a pas collision.
- Utilisation : Idéal pour Boîtes vs Boîtes ou Polyèdres simples.

Méthode 2 : GJK (Gilbert-Johnson-Keerthi)

- Philosophie : « La Différence ». On travaille dans l'espace de Minkowski ($A - B$).
- Fonctionnement : On cherche itérativement si l'Origine $(0, 0, 0)$ est contenue dans la forme $A - B$ en construisant un **Simplex** (Tétraèdre) à l'intérieur.
- Utilisation : Le standard pour les formes convexes quelconques (Capsules, Cones, Mesh convexes).

Focus : Comprendre GJK et le Support Mapping

GJK est un algorithme « aveugle ». Il ne connaît pas la géométrie, il pose juste une question à l'objet via une Fonction de Support :

« **Quel est ton point le plus extrême dans la direction $\vec{d}$?** »

$$S_{A-B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$$

Cela permet de découpler l'algorithme de collision (GJK) de la définition géométrique des objets.

- Sphère : Centre + Rayon * $\vec{d}$
- Cube : Coin correspondant au signe de $\vec{d}$

Résumé du Pipeline

1. **Update Position** : $P = P + V * dt$
2. **Broad Phase** : Mise à jour de la grille/Octree -> Liste de paires candidates.
3. **Narrow Phase** :
 - Check AABB (rapide)
 - Check GJK ou SAT (précis) -> Collision OUI/NON
4. **Contact Generation (EPA/Clipping)** : Point de contact, Normale, Profondeur.
5. **Resolution** : Application des impulsions pour séparer les objets.

Session 7 : Révisions et Quiz

Session présenteielle

Révisions et Quiz (20 points).

Session 8 : Physique de la Rotation

💡 L'objectif du jour

Passer de la particule (un point) au corps rigide (une forme). Un corps rigide ne fait pas que se déplacer, il **pivote** autour de son centre de masse.

1. Le Dictionnaire de Traduction

Concept	Translation (Linéaire)	Rotation (Angulaire)
Position	$\vec{r}$ (m)	Angle θ (rad)
Vitesse	$\vec{v}$ (m/s)	Vitesse Angulaire ω (rad/s)
Résistance	Masse m (kg)	Moment d'Inertie I (kg·m ²)
Cause	Force $\vec{F}$ (N)	Couple / Torque τ (N·m)
Loi de Newton	$\vec{F} = m\vec{a}$	$\tau = I\alpha$

💬 2. Le Couple (Torque) : Faire tourner

Le couple τ dépend de où on pousse.

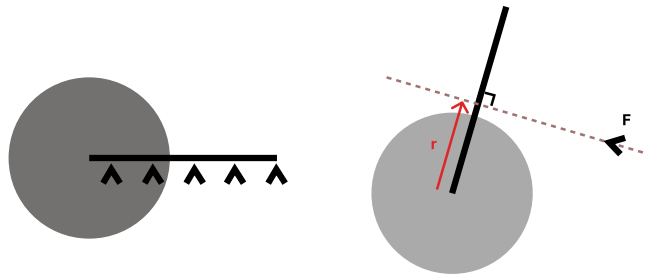


Fig. 15. – Plus on tire loin du centre, plus l'effet de rotation est fort.

En 3D (Vectoriel) : La formule générale utilise un produit vectoriel.

$$\vec{\tau} = \vec{r} \times \vec{F}$$

τ est donc un vecteur perpendiculaire au plan de rotation.

En 2D (Scalaire) : En 2D, la formule se simplifie.

$$\tau = r_x F_y - r_y F_x$$

(Où $\vec{r}$ est le vecteur allant du centre de masse au point d'impact)

3. Le Moment d'Inertie (I ou J)

Le moment d'inertie représente la résistance d'un objet à la rotation.

Il dépend de la répartition de la masse, et de la forme de l'objet. Toujours par rapport à l'axe de rotation bien entendu.

La formule générale est :

$$I = \sum_i m_i \cdot r_i^2$$

ou r_i est la distance au centre de rotation de la masse m_i .

Pour un anneau mince de masse m et de rayon R , on a :

$$I = mR^2$$

, en effet, toute la masse de l'anneau est à la distance fixe R du centre de rotation.

Exercice A : Calcul du moment d'inertie d'un anneau

Retrouver $I = mR^2$ pour un anneau mince de masse m et de rayon R .

1. Paramètres :

- Densité linéaire : $\lambda = \frac{m}{L} = \frac{m}{2\pi R}$ (où L est la circonférence).
- Masse infinitésimale d'un petit segment : $dm = \lambda R d\theta$.
- Distance à l'axe pour tout point : $r = R$.

2. Calcul par intégration : Le moment d'inertie total est la somme (intégrale) des moments infinitésimaux $dI = r^2 dm$.

$$I = \int dI = \int_0^{2\pi} R^2 \cdot (\lambda R d\theta)$$

Sortons les constantes de l'intégrale :

$$I = \lambda R^3 \int_0^{2\pi} d\theta$$

Résolution de l'intégrale :

$$I = \lambda R^3 [\theta]_0^{2\pi}$$

$$I = \lambda R^3 (2\pi - 0) = 2\pi \lambda R^3$$

3. Substitution et simplification : Remplaçons λ par sa définition ($\frac{m}{2\pi R}$) :

$$I = \left(\frac{m}{2\pi R} \right) \cdot 2\pi R^3$$

$$I = mR^2$$

Conclusion : Pour un anneau, toute la masse est à la distance maximale R . C'est l'objet qui offre la plus grande résistance à la rotation pour une masse donnée.

- Rectangle (Largeur w , Hauteur h) :

$$I = \frac{1}{12}m(w^2 + h^2)$$

Pourquoi $\frac{1}{12}$? C'est le résultat mathématique de l'intégration de la masse sur toute la surface du rectangle.

4. Vitesse d'un point sur le corps

C'est l'équation la plus importante pour la détection de collision future. Si un corps se déplace à $\vec{v}_G$ et tourne à ω , la vitesse d'un point P situé à la distance $\vec{r}$ du centre est :

$$\vec{v}_P = \vec{v}_G + (\omega \times \vec{r})$$

En 2D, cela devient :

$$v_{P.x} = v_{G.x} - \omega \cdot r_y$$

$$v_{P.y} = v_{G.y} + \omega \cdot r_x$$

Travaux Pratiques : Le 'Spinning Box'

Objectif

Implémenter un rectangle qui tourne lorsqu'on clique dessus avec la souris (application d'une force à un point précis).

Étape 1 : Mise à jour de la classe RigidBody

Ajoutez les propriétés suivantes à vos objets :

- angle (float)
- angularVelocity (float)
- torque (float)
- Inertia (float) : Calculez-le une seule fois au début via la formule du rectangle.

Étape 2 : La méthode ApplyForce(force, worldPoint)

Cette méthode remplace l'ancien ajout direct à l'accélération :

1. Calculer le bras de levier : $\vec{r} = \text{worldPoint} - \text{positionCentre}$.
2. Ajouter à la force linéaire : $\vec{F}_{\text{total}} += \vec{\text{force}}$.
3. Ajouter au torque : $\tau += (r_x F_y - r_y F_x)$.

Étape 3 : L'intégrateur (Euler Semi-Implicite)

Dans votre boucle update(dt) :

1. $a = \frac{F}{m}$
2. $\alpha = \frac{\tau}{I}$
3. $v += a \cdot dt$
4. $\omega += \alpha \cdot dt$
5. $p += v \cdot dt$
6. $\theta += \omega \cdot dt$
7. Important : Remettre F et tau à zéro pour la frame suivante.

Test de compréhension

« A) L'objet tourne plus vite si je clique au centre. », « B) L'objet tourne plus vite si je clique sur un coin. », « C) L'endroit du clic n'a pas d'importance. »

« Pensez au bras de levier r. »

Session 9 : L'Intégration de Verlet

Qu'est-ce que l'Intégration de Verlet ?

Contrairement à l'intégration d'Euler, l'algorithme de Verlet ne stocke pas explicitement la vitesse d'un objet.

La vitesse est déduite de la différence entre la position actuelle et la position à l'instant précédent. C'est un schéma d'intégration symplectique, ce qui signifie qu'il conserve l'énergie de manière bien plus stable qu'Euler.

💡 Le Concept de Base

Imaginez que vous savez où vous êtes (P_n) et où vous étiez il y a un instant ($P_{\{n-1\}}$). La direction et la distance entre ces deux points représentent votre élan (inertie).

💬 La Formule Mathématique

La position future ($x_{\{n+1\}}$) est calculée à partir de la position actuelle (x_n) et de la précédente ($x_{\{n-1\}}$) :

$$x_{\{n+1\}} = x_n + (x_n - x_{\{n-1\}}) + a \cdot dt^2$$

Où :

- $(x_n - x_{\{n-1\}})$ représente la vitesse (pseudo-vitesse).
- $a \cdot dt^2$ représente l'accélération (comme la gravité).

L'Algorithme en 3 Étapes

Pour chaque objet dans la simulation, on suit cet ordre strict :

1. Calcul de la Vitesse : `vitesse = pos - old_pos`
2. Mise à jour :
 - On sauve la position actuelle : `temp = pos`
 - On calcule la nouvelle : `pos = pos + vitesse + (accel * dt * dt)`
 - On met à jour l'ancienne : `old_pos = temp`
3. Contraintes : On ajuste pos directement (murs, collisions).

Comparaison : Euler vs Verlet. **Euler Explicite :**

- Avantage : Très simple.
- Inconvénient : « Explose » vite. Si on modifie la position d'un objet sans changer sa vitesse, le moteur devient incohérent.

Verlet :

- Avantage : **Inconditionnellement stable**. Si on déplace un objet manuellement (contrainte), sa vitesse s'ajuste d'elle-même à la frame suivante.
- Idéal pour : Les tas de billes, les cordes, les tissus et les chevelures.

Gestion des Collisions

En Verlet, on ne calcule pas de forces de réaction complexes. On utilise la Relaxation de Contrainte.

Si une bille pénètre dans un mur :

- On la téléporte à la surface du mur ($\text{pos.y} = \text{sol}$).
- À la prochaine frame, le calcul $\text{pos} - \text{old_pos}$ donnera une vitesse nulle ou réduite.
- Le système s'auto-équilibre.

$$v_{\text{new}} = \frac{\text{pos}_{\text{corrigée}} - \text{old}_{\text{pos}}}{dt}$$

Annexe : La Méthode Runge-Kutta (RK4)

🔗 Le "Couteau Suisse" de la simulation physique

Dans les sessions précédentes, nous avons utilisé la méthode d'Euler. Elle est simple, mais elle « dérape » vite car elle suppose que la vitesse est constante durant tout le pas de temps.

La méthode de Runge-Kutta d'ordre 4 (RK4) corrige ce problème en étant beaucoup plus astucieuse : au lieu de regarder la pente (dérivée) uniquement au début, elle « sonde » le terrain à 4 endroits différents pour trouver une trajectoire moyenne quasi-parfaite.

1. Le Concept : 4 sondes valent mieux qu'une

Imaginez que vous êtes aveugle et que vous devez traverser une vallée vallonnée en un pas de temps Δt .

- **Euler (k1)** : Vous tendez le pied, sentez la pente actuelle, et faites un grand saut dans cette direction. → *Risqué*.
- **RK4** : Vous envoyez des éclaireurs virtuels avant de bouger.
 1. **k1 (Le Départ)** : Pente au début (comme Euler).
 2. **k2 (Le Milieu A)** : On utilise la pente $k1$ pour avancer jusqu'à la moitié du pas ($\Delta \frac{t}{2}$) et on regarde la pente là-bas.
 3. **k3 (Le Milieu B)** : On se méfie de $k2$. On repart du début, mais cette fois on utilise la pente $k2$ pour aller au milieu. On regarde la nouvelle pente.
 4. **k4 (L'Arrivée)** : On utilise la pente $k3$ pour aller jusqu'à la fin du pas (Δt) et on regarde la pente finale.

Le Grand Final : On fait une moyenne pondérée de ces 4 pentes. On donne plus de poids aux pentes du milieu ($k2$ et $k3$).

$$\text{Pente Finale} = \frac{k_1 + 2k_2 + 2k_3 + k_4}{6}$$

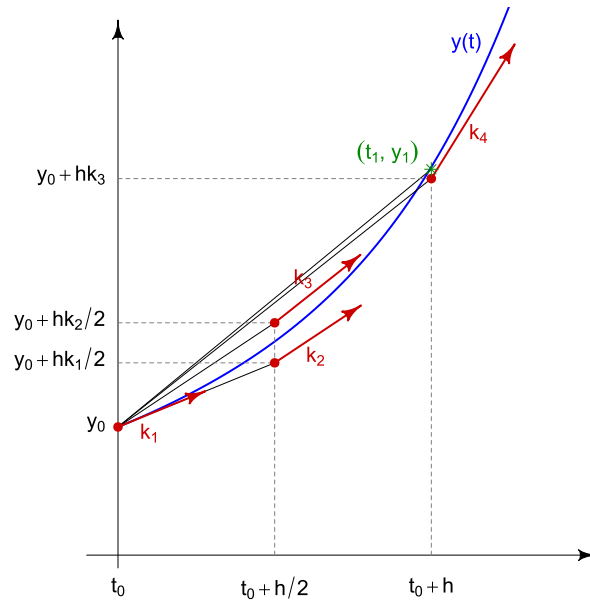


Fig. 16. – Runge Kutta : 4 pentes pour une meilleure trajectoire

2. Les Mathématiques (L'État du Système)

Pour implémenter RK4 proprement, nous devons regrouper nos variables (Position et Vitesse) dans un seul vecteur que nous appellerons l'État (Y).

Si on a un système physique simple :

$$Y = \begin{pmatrix} x \\ v \end{pmatrix}$$

$$\dot{Y} = f(t, Y) = \begin{pmatrix} v \\ a \end{pmatrix}$$

La dérivée de l'état (le changement), c'est la vitesse et l'accélération.

Voici l'algorithme complet pour un pas de temps h (ou Δt) :

Algorithme RK4 pour un pas h

Soit la fonction `eval(état)` qui retourne la dérivée [vitesse, accélération].

1. k_1 (**Début**) = `eval(état_actuel)`
2. k_2 (**Milieu**) = `eval(état_actuel +  $k_1 \times \frac{h}{2}$ )`
3. k_3 (**Milieu**) = `eval(état_actuel +  $k_2 \times \frac{h}{2}$ )`
4. k_4 (**Fin**) = `eval(état_actuel +  $k_3 \times h$ )`

$$\text{Nouvel État} = \text{état_actuel} + \frac{k_1 + 2k_2 + 2k_3 + k_4}{6} \times h$$

3. Implémentation (Pseudo-Code / JavaScript)

Contrairement à Euler où l'on écrit `pos += vel * dt` directement, RK4 nécessite une structure plus organisée.

```

// Définition d'un État
struct State {
    Vector3 position;
    Vector3 velocity;
};

// La dérivée de l'état, c'est ce qui change
struct Derivative {
    Vector3 d_position; // C'est la vitesse
    Vector3 d_velocity; // C'est l'accélération (Forces / Masse)
};

// Fonction qui calcule les forces et retourne la dérivée
function evaluate(initialState, t, dt, derivative) {
    // 1. Estimer l'état futur basé sur la dérivée précédente
    State state = initialState;
    state.position += derivative.d_position * dt;
    state.velocity += derivative.d_velocity * dt;

    // 2. Calculer les forces à cet endroit/vitesse là
    // (Exemple: Gravité + Vent + Ressort)
    Vector3 forces = calculateForces(state.position, state.velocity);
    Vector3 acceleration = forces / mass;

    // 3. Retourner la nouvelle dérivée
    return new Derivative(state.velocity, acceleration);
}

// BOUCLE PRINCIPALE (INTÉGRATEUR)
function integrateRK4(state, t, dt) {
    // a. Préparer les 4 échantillons
    Derivative a = evaluate(state, t, 0.0, new Derivative()); // k1
    Derivative b = evaluate(state, t, dt*0.5, a); // k2
    Derivative c = evaluate(state, t, dt*0.5, b); // k3
    Derivative d = evaluate(state, t, dt, c); // k4

    // b. Moyenne pondérée pour la position (dxdt)
    Vector3 dxdt = (a.d_pos + 2*(b.d_pos + c.d_pos) + d.d_pos) * 1/6;

    // c. Moyenne pondérée pour la vitesse (dvdt)
    Vector3 dvdt = (a.d_vel + 2*(b.d_vel + c.d_vel) + d.d_vel) * 1/6;

    // d. Mise à jour finale
    state.position += dxdt * dt;
    state.velocity += dvdt * dt;
}

```

Session 10 : Encore plus de forces

1. Les Ressorts : La Loi de Hooke

Un ressort est une force qui essaie toujours de ramener un objet à une position d'équilibre.

La Formule :

$$\vec{F} = -k \cdot (L - L_0) \cdot \hat{u}$$

- k : Raideur du ressort (Stiffness).
- L : Longueur actuelle.
- L_0 : Longueur au repos.
- $\hat{u}$: Direction (unitaire).

Le problème de la stabilité : Si on utilise la méthode d'Euler avec un k élevé, le système « explose ». C'est l'équation d'un oscillateur harmonique qui accumule de l'énergie à chaque pas de temps.

Solution 1 : Le Damping (Amortissement) On ajoute une force qui s'oppose à la vitesse pour dissiper l'énergie :

$$\vec{F}_{\text{damping}} = -b \cdot \vec{v}$$

Solution 2 : Verlet L'intégration de Verlet est naturellement plus stable pour les systèmes de ressorts.

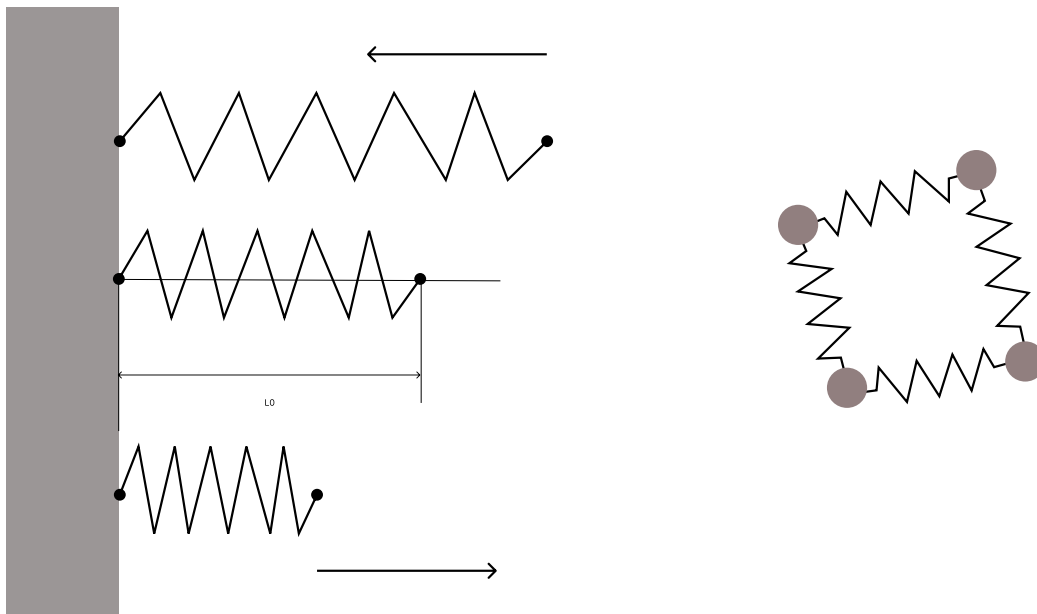


Fig. 17. – La Loi de Hooke : Force de rappel proportionnelle à l'étirement

2. Les Contraintes de Verlet (Linking Particles)

Au lieu de simuler des ressorts complexes, on peut utiliser l'intégration de Verlet pour créer des « bâtons » rigides entre des particules.

Principe :

1. On met à jour toutes les particules avec Verlet (gravité, etc.).
2. Pour chaque contrainte (bâton), on calcule l'écart entre la distance actuelle et la distance souhaitée.
3. On déplace les deux particules pour corriger la moitié de l'écart chacune.
4. On répète l'opération plusieurs fois (itérations) pour converger.

Avantage : Pas besoin de calculer des forces. On manipule directement les positions. C'est la base des moteurs PBD (Position Based Dynamics).

3. La Gravité Universelle

Newton a découvert que **tous** les objets s'attirent mutuellement.

$$\vec{F} = G \cdot \frac{m_1 m_2}{r^2} \cdot \hat{u}$$

- G : Constante gravitationnelle ($6.674 \times 10^{-11} \text{N} \cdot \frac{\text{m}^2}{\text{kg}^2}$).
- r : Distance entre les centres.
- $\hat{u}$: Direction de l'attraction.

Pourquoi on ne ressent pas l'attraction entre deux personnes ? Parce que G est minuscule. Il faut une masse planétaire pour que la force soit perceptible.

Dans un jeu vidéo : On simule un système solaire en appliquant cette force entre chaque paire d'objets.

TP : Le Système Solaire

Objectif :

1. Créer un soleil massif au centre.
2. Ajouter plusieurs planètes avec des vitesses initiales perpendiculaires au soleil.
3. Appliquer la gravité universelle entre chaque paire.
4. Observer les orbites.

Défi : Trouver la bonne vitesse initiale pour qu'une planète orbite en cercle parfait.

$$v = \sqrt{G \frac{M}{r}}$$

Session 11 : Moteurs Physiques & Middleware

1. Pourquoi utiliser un moteur physique ?

Jusqu'à présent, nous avons codé notre propre moteur. C'est excellent pour apprendre, mais en production, on utilise des moteurs existants.

Pourquoi ?

- **Complexité** : Gestion des contacts multiples, stabilité, optimisation.
- **Temps** : Un moteur professionnel représente des décennies de R&D.
- **Performance** : Optimisé pour des milliers d'objets.

2. Concepts Avancés des Moteurs Modernes

A. Rigid Body vs. Soft Body

- **Rigid Body** : Objet indéformable (boîte, sphère). C'est ce qu'on a codé.
- **Soft Body** : Objet déformable (tissu, gelée). Beaucoup plus complexe.

B. Continuous Collision Detection (CCD)

- **Problème** : Si un objet va très vite, il peut « traverser » un mur entre deux frames (Tunneling).
- **Solution** : Au lieu de checker la position à t et $t + \Delta t$, on calcule la trajectoire complète (raycast entre les deux positions).

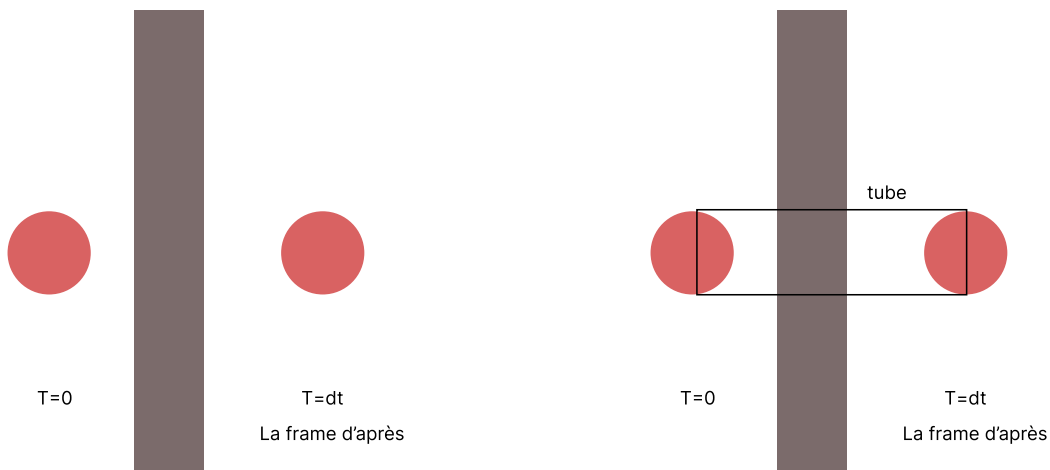


Fig. 18. – Continuous Collision Detection : Éviter le Tunneling

C. Sleeping

- **Concept** : Si un objet est immobile depuis plusieurs frames, on arrête de le simuler. Il « dort ».
- **Réveil** : Si un autre objet le touche, il se réveille.
- **Bénéfice** : Économie CPU massive pour les scènes statiques.

3. Comparatif des Moteurs Physiques

Moteur	Langage	Usage
PhysX	C++	AAA Games (Unity, Unreal default)

Moteur	Langage	Usage
Havok	C++	AAA Games (Half-Life 2, Zelda TotK)
Jolt Physics	C++	Modern, open-source, Horizon Forbidden West
Box2D	C++	2D Games (Angry Birds, Terraria)
Cannon.js	JavaScript	Three.js (simple, legacy)
Ammo.js	JavaScript (WASM)	Three.js (Bullet port)
Rapier	Rust/WASM	Modern, fast, Three.js recommended

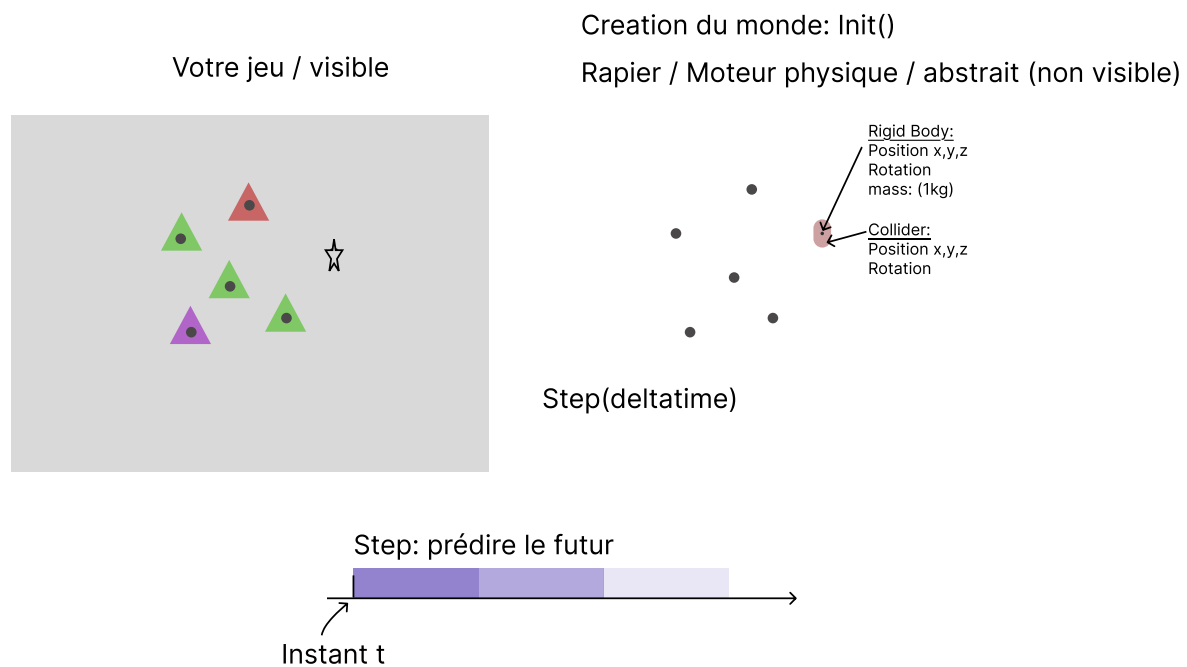


Fig. 19. – Architecture d'un moteur physique moderne

4. Le Déterminisme

Un moteur physique est **déterministe** si, pour les mêmes inputs, il produit exactement les mêmes outputs.

Pourquoi c'est important ?

- **Multijoueur** : Si tous les clients simulent la même scène, ils doivent voir la même chose.
- **Replay** : Pour rejouer une action, il suffit de stocker les inputs, pas toutes les positions.

Pièges :

- L'ordre des opérations peut varier.

- Les calculs flottants peuvent différer selon le CPU.
- Rapier est déterministe par design.

5. L'Architecture ECS (Entity-Component-System)

C'est le pattern moderne pour organiser un jeu vidéo.

- **Entity** : Un simple ID (ex: entity_42).
- **Component** : Données pures (ex: Position, Rigidbody, Mesh).
- **System** : Logique qui opère sur les composants (ex: PhysicsSystem, RenderSystem).

Avantage : Extrêmement flexible. On peut ajouter/enlever des composants à la volée sans changer le code des systèmes.

Session 12 : RigidBodyes & Colliders

💡 Introduction

Maintenant que nous comprenons les concepts, nous allons utiliser un vrai moteur physique : Rapier.js.

Rapier est un moteur 3D écrit en Rust, compilé en WebAssembly, et utilisable en JavaScript. C'est le moteur recommandé pour Three.js.

1. Les RigidBodyes

Un RigidBody dans Rapier est l'objet principal. Il a :

- Une position (translation + rotation).
- Une vitesse (linéaire et angulaire).
- Une masse.
- Un type : Dynamic, Fixed, Kinematic.

Types de RigidBodyes :

- **Dynamic** : Réagit aux forces et collisions (comme nos boules).
- **Fixed** : Immobile. Les autres objets rebondissent contre lui (comme nos murs).
- **Kinematic** : Contrôlé manuellement par le code. Ignore les forces mais pousse les Dynamic.

Créer un RigidBody dans Rapier.

```
import RAPIER from '@dimforge/rapier3d';

// 1. Créer le monde physique
const world = new RAPIER.World({ x: 0, y: -9.81, z: 0 });

// 2. Créer un RigidBody dynamique
const rigidBodyDesc = RAPIER.RigidBodyDesc.dynamic()
  .setTranslation(0, 5, 0);
const rigidBody = world.createRigidBody(rigidBodyDesc);

// 3. Créer un Collider et l'attacher
const colliderDesc = RAPIER.ColliderDesc.ball(1.0);
const collider = world.createCollider(colliderDesc, rigidBody);
```

2. Les Colliders

Le Collider définit la **forme** de l'objet pour la détection de collision. Un RigidBody peut avoir plusieurs Colliders.

Formes disponibles :

- ball(radius) : Sphère.
- cuboid(hx, hy, hz) : Boîte (half-extents).
- capsule(halfHeight, radius) : Capsule.
- cylinder(halfHeight, radius) : Cylindre.
- convexHull(points) : Forme convexe personnalisée.
- trimesh(vertices, indices) : Mesh quelconque (statique uniquement).

La Boucle de Simulation.

```
function animate() {
  requestAnimationFrame(animate);

  // Stepping the world
  world.step();

  // Sync Three.js meshes with Rapier bodies
  for (const { mesh, rigidBody } of objects) {
    const position = rigidBody.translation();
    const rotation = rigidBody.rotation();

    mesh.position.set(position.x, position.y, position.z);
    mesh.quaternion.set(rotation.x, rotation.y, rotation.z, rotation.w);
  }

  renderer.render(scene, camera);
}
```

Session 13 : Joints, Moteurs & Raycasts

1. Les Joints (Articulations)

Les joints relient deux RigidBodyes entre eux, limitant leur mouvement relatif.

Types de Joints :

- **Fixed Joint** : Les deux corps sont rigidement liés (comme un soudage).
- **Revolute Joint** : Rotation autour d'un axe (comme une charnière de porte).
- **Prismatic Joint** : Translation le long d'un axe (comme un tiroir).
- **Spherical Joint** : Rotation libre dans toutes les directions (comme une épaule).

Configuration : Chaque joint nécessite des **ancres** (anchors) : la position du joint sur chaque corps, dans les coordonnées locales de chaque corps.

Créer une charnière (Revolute Joint).

```
const jointDesc = RAPIER.JointData.revolute(
  { x: 0, y: 1, z: 0 }, // Anchor on body A (local)
  { x: 0, y: -1, z: 0 }, // Anchor on body B (local)
  { x: 0, y: 1, z: 0 } // Axis of rotation (world)
);
const joint = world.createJoint(jointDesc, bodyA, bodyB);
```

2. Les Moteurs et Limites

On peut ajouter des **moteurs** aux joints pour les faire tourner activement.

```
// Configurer un moteur sur un revolute joint
joint.configureMotorVelocity(targetVelocity, stiffness, damping);
```

On peut aussi ajouter des **limites** (min/max angle) pour empêcher une rotation excessive (comme un coude qui ne peut pas tourner à 360°).

3. Les Raycasts

Un raycast projette un rayon invisible dans la scène et retourne le premier objet touché.

Utilisations :

- Détection du sol (Character Controller).
- Visée (FPS).
- Interaction (cliquer sur un objet).
- Capteurs de voiture (autonomous driving).

Raycast dans Rapier.

```
const ray = new RAPIER.Ray(
  { x: 0, y: 10, z: 0 }, // Origin
  { x: 0, y: -1, z: 0 } // Direction
);
const hit = world.castRay(ray, 20.0, true);

if (hit) {
  console.log("Touché à la distance:", hit.timeOfImpact);
}
```

```
console.log("Collider touché:", hit.collider);  
}
```

TP : La Machine de Siège

Objectif : Construire une catapulte fonctionnelle.

1. Créer une base fixe.
2. Créer un bras avec un revolute joint.
3. Ajouter un moteur au joint.
4. Lancer un projectile.

Session 14 : Les Solveurs de Contraintes

1. PBD vs Impulse Based

Il existe deux grandes approches pour résoudre les contraintes et collisions :

A. Position Based Dynamics (PBD)

- **Philosophie** : Corriger directement les positions.
- **Méthode** : Si deux objets se chevauchent, on les sépare manuellement.
- **Avantage** : Simple, stable visuellement.
- **Inconvénient** : La physique n'est pas « vraie » (les vitesses ne sont pas correctes).
- **Utilisé par** : Verlet, PhysX (Soft Body), BeamNG.

B. Impulse Based

- **Philosophie** : Calculer les impulsions nécessaires pour respecter les contraintes.
- **Méthode** : On calcule j (comme dans le TP collision) pour chaque contact.
- **Avantage** : Physiquement correct, conserve la quantité de mouvement.
- **Inconvénient** : Plus complexe, peut nécessiter plusieurs itérations.
- **Utilisé par** : Box2D, Rapiere, Havok, PhysX (Rigid Body).

2. Le Concept de Contrainte

Une contrainte est une équation qui doit être satisfaite à chaque frame.

Exemple : Contrainte de distance Deux particules A et B doivent rester à une distance L .

$$C = \|\overrightarrow{AB}\| - L = 0$$

Si $C \neq 0$, il faut appliquer une correction.

3. L'Algorithme PGS (Projected Gauss-Seidel)

C'est l'algorithme utilisé par les moteurs modernes pour résoudre les impulsions.

Principe :

1. Pour chaque contact, calculer l'impulsion nécessaire.
2. L'appliquer.
3. Mais cette application a modifié les vitesses des autres contacts !
4. Donc on recommence (itération).
5. Après N itérations, le système converge vers une solution stable.

Le nombre d'itérations est un paramètre clé :

- Trop peu : Les objets s'enfoncent (soft).
- Trop : Le CPU surchauffe.
- Typique : 4 à 8 itérations.

4. Techniques Avancées de Stabilité

A. Warm Starting

- **Idée** : Au lieu de repartir de zéro à chaque frame, on réutilise les impulsions de la frame précédente.
- **Bénéfice** : Convergence beaucoup plus rapide (1-2 itérations suffisent).
- **C'est comme « préchauffer » le solveur.**

B. Sleeping

- Déjà vu : les objets immobiles ne sont plus simulés.

C. Position Correction (Baumgarte)

- Même avec les impulsions, les objets peuvent légèrement s'enfoncer.
- On ajoute une petite correction de position pour les séparer.
- $$\text{correction} = \text{penetration} \times \text{BaumgarteFactor} \times \vec{n}$$
- Le facteur est généralement 0.2 (20% de correction par frame).

5. Paramétrer un Solveur

Les moteurs physiques exposent plusieurs paramètres :

- **Velocity Threshold** : En dessous de cette vitesse, l'objet est considéré comme immobile.
- **Restitution Threshold** : En dessous de cette vitesse de collision, pas de rebond.
- **Max Penetration** : Distance maximale d'enfoncement autorisée.
- **Solver Iterations** : Nombre d'itérations du PGS.

Le jeu du développeur : Trouver le bon équilibre entre précision et performance.

Session 15 : Cinématique Inverse (IK)

1. Forward Kinematics vs Inverse Kinematics

Forward Kinematics (FK)

- On donne les angles des articulations, et on calcule la position de la main.
- $\vec{p} = f(\theta_1, \theta_2, \theta_3, \dots)$
- C'est ce qu'on fait depuis le début : on donne une rotation, le moteur calcule la position.

Inverse Kinematics (IK)

- On donne la position désirée de la main, et on calcule les angles des articulations.
- $\theta_1, \theta_2, \theta_3, \dots = f^{-1}(\vec{p})$
- C'est ce qu'on veut faire : « Je veux que ma main touche cette pomme, calcule comment plier mon bras. »

Pourquoi c'est difficile ?

- Il y a souvent **plusieurs solutions** (le bras peut se plier de plusieurs façons).
- Parfois il n'y a **aucune solution** (la cible est trop loin).
- L'équation f^{-1} n'a pas de solution analytique simple.

2. Algorithme 1 : CCD (Cyclic Coordinate Descent)

C'est l'algorithme le plus simple à comprendre et à coder.

Principe : On ajuste les articulations une par une, en partant de l'extrémité (la main) vers la base (l'épaule).

Boucle :

1. Prendre la dernière articulation (le poignet).
2. Calculer l'angle entre (articulation → cible) et (articulation → main).
3. Tourner l'articulation de cet angle.
4. Passer à l'articulation précédente (le coude).
5. Répéter jusqu'à la base.
6. Recommencer la boucle entière plusieurs fois (itérations).

Avantage : Très simple, converge rapidement. **Inconvénient :** Les mouvements ne sont pas naturels (les articulations proches de la base bougent trop).

3. Algorithme 2 : FABRIK (Forward And Backward Reaching Inverse Kinematics)

C'est l'algorithme le plus populaire pour l'animation en temps réel.

Principe : Au lieu de calculer des angles, on manipule directement les positions des articulations.

Boucle :

1. **Forward :** Partir de la main et remonter vers l'épaule. Chaque articulation est déplacée pour être à la bonne distance de la suivante.
2. **Backward :** Partir de l'épaule et redescendre vers la main. Chaque articulation est repositionnée pour respecter les longueurs des segments.
3. Répéter jusqu'à convergence.

Avantage : Mouvements naturels, très rapide, facile à implémenter. **Inconvénient :** Ne gère pas les limites d'angles directement (il faut post-traiter).

4. IK et Physique : L'Animation Procédurale

L'IK devient magique quand on la combine avec la physique.

Exemples :

- **Foot Placement :** Les pieds du personnage s'adaptent au terrain irrégulier. On lance un raycast vers le sol, et l'IK place le pied exactement sur la surface.
- **Active Ragdoll :** Un personnage ragdoll (corps mou) qui essaie de maintenir une pose via IK. C'est ce qui donne des morts réalistes dans les jeux modernes.
- **Hand Grab :** La main du personnage attrape un objet précis dans le monde.

Session 16 : Les Systèmes de Particules et le GPU

1. Architecture d'un Système de Particules

Un système de particules simule un grand nombre de petits objets (pluie, feu, fumée, étincelles).

Structure d'une particule :

- Position, Vitesse, Accélération
- Durée de vie (lifetime)
- Couleur, Taille
- Masse (optionnel)

L'Émetteur (Emitter) :

- Génère de nouvelles particules à un certain taux (ex: 1000/s).
- Définit les paramètres initiaux (position, vitesse, direction).
- Peut avoir différentes formes (point, cercle, sphère, cone).

Le Pool (Object Pooling) :

- On ne crée/détruit pas de particules à chaque frame (trop lent).
- On pré-alloue un tableau fixe de particules.
- Quand une particule « meurt », on la recycle.

2. Le Cycle de Vie d'une Particule

1. **Birth** : L'émetteur crée la particule avec des paramètres initiaux.
2. **Update** : Chaque frame, on applique les forces (gravité, drag), on met à jour la position.
3. **Render** : On dessine la particule (souvent un billboard, un quad qui fait face à la caméra).
4. **Death** : Quand `lifetime <= 0`, la particule est désactivée et recyclée.

3. Interactions Physiques

Gravité : $\vec{a} = \vec{g}$ (constant). **Drag** : $\vec{F} = -k \cdot \vec{v}$ (ralentit les particules). **Collisions simplifiées** : On ne fait pas de vraies collisions (trop cher pour 10000 particules). On utilise des plans ou des champs de force.

Exemple : Rebond sur le sol

```
if (particle.position.y < 0) {
    particle.position.y = 0;
    particle.velocity.y *= -restitution; // 0.5 = rebond mou
}
```

4. Optimisation : Le GPU et les Compute Shaders

Pour simuler des centaines de milliers de particules, le CPU ne suffit plus. On déplace le calcul sur le GPU.

Pourquoi le GPU est-il parfait pour ça ?

- Les particules sont **indépendantes** (pas de collisions entre elles).
- Le GPU excelle dans le traitement parallèle (des milliers de cœurs).

- Les données restent sur le GPU (pas de transfert CPU $\leftrightarrow$ GPU).

Compute Shader :

- Un programme qui s'exécute sur le GPU mais qui n'est pas un shader de rendu.
- On stocke les particules dans un SSBO (Shader Storage Buffer Object).
- Le compute shader lit et modifie les positions/vitesses directement dans le buffer.

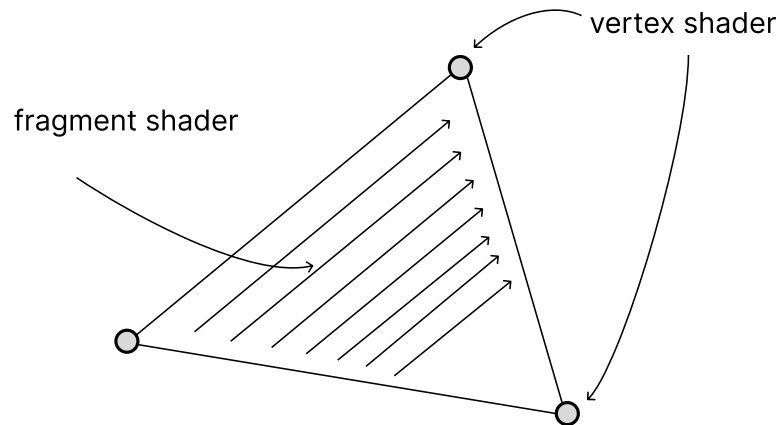
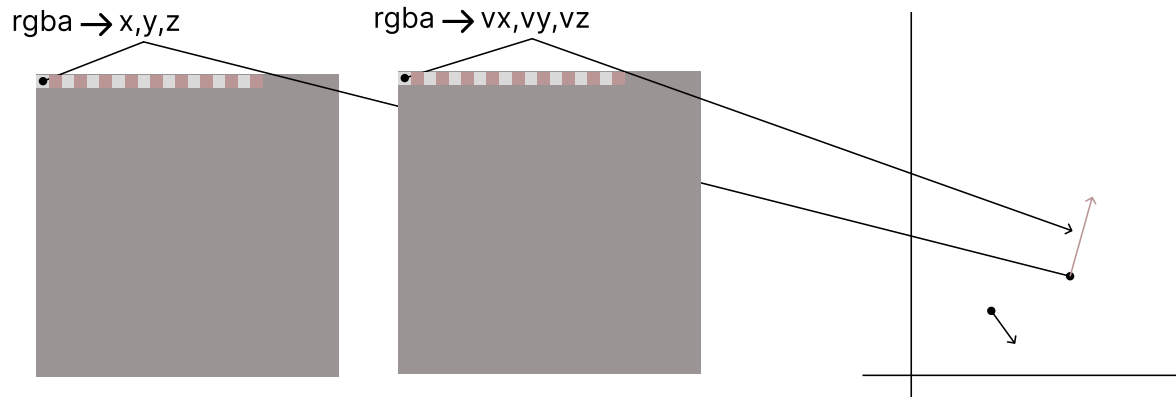


Fig. 20. – Architecture GPU : Compute Shader pour particules

Pipeline :

1. CPU envoie les paramètres (gravité, émission) au GPU.
2. Compute Shader met à jour les particules (position, vitesse, lifetime).
3. Vertex Shader génère les billboards.
4. Fragment Shader dessine les particules.

Résultat : 1 000 000 de particules à 60 FPS sur un GPU moderne.

Session 17 : Character Controllers

1. Le Choix Fondamental : DCC vs KCC

Dynamic Character Controller (DCC)

- Le personnage est un Rigidbody standard.
- Il subit les forces, les collisions, la gravité.
- **Avantage** : Interagit naturellement avec le monde (se fait pousser, tombe, etc.).
- **Inconvénient** : Difficile à contrôler précisément. Le personnage peut glisser, tomber, être poussé par d'autres objets. Pas de « game feel ».

Kinematic Character Controller (KCC)

- Le personnage est un Rigidbody kinematic.
- On contrôle manuellement sa position. La physique ne le pousse pas.
- **Avantage** : Contrôle total. Le personnage fait exactement ce qu'on veut.
- **Inconvénient** : Il faut coder manuellement les collisions, la gravité, les pentes.

La norme dans l'industrie : KCC. Les jeux AAA utilisent presque toujours un KCC pour le joueur.

2. L'Algorithme 'Move and Slide'

C'est l'algorithme standard pour un KCC.

Principe :

1. On calcule le mouvement souhaité (input + gravité + vitesse).
2. On tente de déplacer le personnage de ce vecteur.
3. Si on touche un mur, on **glisse** le long du mur au lieu de s'arrêter.

Mathématique : Projection et Rejection

- Soit $\vec{v}$ la vitesse et $\vec{n}$ la normale du mur.
- **Projection** : La composante dans le mur (à annuler).

$$\vec{v}_{\text{proj}} = (\vec{v} \cdot \vec{n})\vec{n}$$

- **Rejection** : La composante le long du mur (à conserver).

$$\vec{v}_{\text{rej}} = \vec{v} - \vec{v}_{\text{proj}}$$

- Nouvelle vitesse : $\vec{v}' = \vec{v}_{\text{rej}}$

C'est pour ça qu'on « glisse » le long des murs dans les jeux !

3. Le Problème du 'Corner Trap'

Quand on glisse le long de deux murs en même temps (un coin), le personnage peut se bloquer.

Solution : On ne traite qu'une collision à la fois. On trie les collisions par profondeur, on résout la plus profonde, puis on re-tente le mouvement.

4. Détection du Sol

Pour savoir si le personnage est au sol, on lance un raycast vers le bas.

Méthodes :

- **Raycast simple** : Un rayon depuis le centre vers le bas. Simple mais peut rater les bords.
- **Shapecast** : On projette la forme entière (capsule) vers le bas. Plus précis.
- **Multi-raycast** : Plusieurs rayons répartis sous les pieds.

Critère : Si la distance au sol est inférieure à un seuil (ex: 0.1m), le personnage est « au sol » et peut sauter.

5. Game Feel : Coyote Time et Jump Buffering

Ces deux techniques transforment un saut « technique » en un saut « fun ».

Coyote Time :

- Après avoir quitté une plateforme, le joueur peut encore sauter pendant 0.1s.
- **Pourquoi ?** Parce que le cerveau humain a du mal à accepter qu'on ne puisse pas sauter quand on vient de marcher au bord.

Jump Buffering :

- Si le joueur appuie sur « saut » juste **avant** d'atterrir, le saut s'enregistre et s'exécute dès qu'on touche le sol.
- **Pourquoi ?** Parce que appuyer trop tôt ne devrait pas être puni.

Ces deux techniques sont la différence entre un jeu « rigide » et un jeu « fun ».

TP : FPS Controller avec Rapier

Objectif : Implémenter un contrôleur FPS en mode kinematic avec Rapier.

1. Créer un RigidBody kinematic (capsule).
2. Lire les inputs (WASD + souris).
3. Calculer la vitesse souhaitée.
4. Utiliser `world.castRay` pour détecter le sol.
5. Implémenter le Move and Slide.
6. Ajouter le Coyote Time et le Jump Buffering.

Session 18 : Physique des Véhicules

1. Le Modèle 'Raycast Vehicle'

Pour les jeux arcade, on ne simule pas les vraies roues (trop complexe). On utilise le modèle « Raycast Vehicle ».

Anatomie :

- Un **châssis** (RigidBody dynamique).
- Des **roues invisibles** : Chaque roue est un raycast vers le bas depuis le châssis.

Pourquoi des raycasts ?

- Pas besoin de simuler des RigidBodies pour les roues.
- Le raycast détecte le sol, et on applique les forces manuellement.
- Beaucoup plus stable et performant que des vraies roues.

2. La Suspension

Chaque roue simule un ressort (suspension).

Calcul :

1. Lancer un raycast depuis la roue vers le bas.
2. Si on touche le sol à la distance d :
 - Compression : $c = 1 - \frac{d}{\text{maxSuspensionLength}}$
 - Force : $F = k \cdot c - d \cdot \text{damping} \cdot v_{\text{suspension}}$
3. Appliquer la force au châssis au point de contact.

Résultat : La voiture « flotte » au-dessus du sol, maintenue par 4 ressorts invisibles.

3. La Tenue de Route (Tire Grip)

Le pneu exerce des forces dans deux directions :

Force Longitudinale (Accélération/Freinage) :

- On applique un couple moteur à la roue.
- Le pneu « pousse » le sol vers l'arrière, la voiture avance.

Force Latérale (Virage) :

- Quand on tourne le volant, les roues avant changent de direction.
- Le pneu exerce une force latérale qui fait tourner la voiture.
- **Grip** : Si le grip est faible, la voiture dérape (drift).

Le Slip Angle : C'est l'angle entre la direction du pneu et la direction réelle du mouvement.

- Petit angle : La voiture tourne normalement.
- Grand angle : La voiture dérape (drift).

TP : Transformer un Cube en Véhicule

Objectif : Créer un véhicule drivable avec Rapier.

1. Créer un châssis (boîte dynamique).

2. Ajouter 4 raycasts (roues).
3. Implémenter la suspension (ressort + damping).
4. Ajouter l'accélération (force longitudinale).
5. Ajouter la direction (force latérale sur les roues avant).
6. Tester sur un terrain off-road (pentes, bosses).

Défi : Trouver les bons paramètres de suspension pour que la voiture ne bascule pas dans les virages.

Session 19 : Physique & IA

1. Le Besoin : Mouvement Organique

Les animations pré-enregistrées (keyframes) sont rigides. Si un personnage doit s'adapter à un terrain inconnu, on ne peut pas tout pré-enregistrer.

Solution : Utiliser la physique + l'IA pour générer du mouvement en temps réel.

Deux approches dominent :

- **Reinforcement Learning (RL)**
- **Neuroevolution (Genetic Algorithms)**

2. Reinforcement Learning (RL)

La Boucle RL :

1. **Agent :** Le personnage (ex: un robot).
2. **Environment :** Le monde physique (Rapier).
3. **State :** Ce que le robot « voit » (capteurs, positions, vitesses).
4. **Action :** Ce que le robot fait (appliquer des forces/torques).
5. **Reward :** Une note qui dit si l'action était bonne ou mauvaise.

Le Cerveau : Un Réseau de Neurones (Neural Network).

- Input : L'état (State).
- Output : Les actions (forces à appliquer).
- Entraînement : Le réseau ajuste ses poids pour maximiser la reward.

Exemple : Apprendre à marcher

- Reward : Avancer le plus loin possible.
- Pénalité : Tomber.
- Après des millions d'itérations, le robot apprend à marcher tout seul.

3. Neuroevolution (Genetic Algorithms)

C'est une alternative à RL, inspirée de l'évolution biologique.

Principe :

1. Créer une population de N agents (ex: 50 voitures), chacune avec un réseau de neurones aléatoire.
2. Les faire rouler dans le circuit.
3. Évaluer : Celles qui vont le plus loin ont le meilleur « fitness ».
4. **Sélection :** Garder les meilleures, tuer les autres.
5. **Crossover :** Mélanger les réseaux des meilleures.
6. **Mutation :** Ajouter de petites modifications aléatoires.
7. Nouvelle génération → Retour à l'étape 2.

Avantage : Plus simple à implémenter que RL. Pas besoin de gradients. **Inconvénient :** Plus lent à converger. Moins efficace pour des problèmes complexes.

4. Les Capteurs Physiques comme Inputs

Le réseau de neurones a besoin d'informations sur le monde. On utilise des capteurs physiques :

Raycasts (Vision) :

- Comme les capteurs d'une voiture autonome.
- N rayons vers l'avant à différents angles.
- Chaque rayon retourne la distance au premier obstacle.
- C'est l'« œil » de l'IA.

Vitesse :

- La vitesse actuelle du véhicule (vector).

Orientation :

- L'angle entre le véhicule et la direction de la piste.

Exemple de réseau simple :

- Input : 5 raycasts + vitesse + orientation = 7 inputs.
- Hidden : 8 neurones.
- Output : 2 (accélération + direction).
- Total : $7 \times 8 + 8 \times 2 = 72$ poids à optimiser.

TP : Voiture Autonome

Objectif : Créer une voiture qui apprend à naviguer un circuit toute seule.

1. Créer un circuit (murs, checkpoints).
2. Créer N voitures avec un Raycast Vehicle.
3. Chaque voiture a un réseau de neurones simple.
4. Inputs : 5 raycasts + vitesse.
5. Outputs : accélération + direction.
6. Fitness : Distance parcourue avant de crasher.
7. Implémenter la sélection, le crossover et la mutation.
8. Lancer la simulation et observer l'évolution !

TP Guide : Unity & IA

🔔 Objectif

Implémenter une voiture autonome dans Unity en utilisant un réseau de neurones simple et un algorithme génétique.

1. Le Réseau de Neurones (C\#)

Voici une classe C# simple pour un réseau de neurones :

```
using System;
using UnityEngine;

public class NeuralNetwork
{
    private int[] layers;
    private float[][] neurons;
    private float[][][] weights;

    public NeuralNetwork(int[] layers)
    {
        this.layers = layers;
        InitNeurons();
        InitWeights();
    }

    private void InitNeurons()
    {
        neurons = new float[layers.Length][];
        for (int i = 0; i < layers.Length; i++)
            neurons[i] = new float[layers[i]];
    }

    private void InitWeights()
    {
        weights = new float[layers.Length - 1][][];
        for (int i = 0; i < layers.Length - 1; i++)
        {
            weights[i] = new float[neurons[i + 1].Length][];
            for (int j = 0; j < neurons[i + 1].Length; j++)
            {
                weights[i][j] = new float[neurons[i].Length];
                for (int k = 0; k < neurons[i].Length; k++)
                    weights[i][j][k] = Random.Range(-1f, 1f);
            }
        }
    }

    public float[] FeedForward(float[] inputs)
    {
        for (int i = 0; i < inputs.Length; i++)
            neurons[0][i] = inputs[i];
    }
}
```

```

    for (int i = 1; i < layers.Length; i++)
    {
        for (int j = 0; j < neurons[i].Length; j++)
        {
            float value = 0f;
            for (int k = 0; k < neurons[i - 1].Length; k++)
                value += weights[i - 1][j][k] * neurons[i - 1][k];
            neurons[i][j] = Mathf.Tanh(value); // Activation
        }
    }
    return neurons[neurons.Length - 1];
}

public NeuralNetwork Copy()
{
    NeuralNetwork nn = new NeuralNetwork(layers);
    for (int i = 0; i < weights.Length; i++)
        for (int j = 0; j < weights[i].Length; j++)
            for (int k = 0; k < weights[i][j].Length; k++)
                nn.weights[i][j][k] = weights[i][j][k];
    return nn;
}

public void Mutate(float rate)
{
    for (int i = 0; i < weights.Length; i++)
        for (int j = 0; j < weights[i].Length; j++)
            for (int k = 0; k < weights[i][j].Length; k++)
                if (Random.Range(0f, 1f) < rate)
                    weights[i][j][k] += Random.Range(-0.5f, 0.5f);
}
}

```

2. Collecter les Données des Capteurs

On utilise des raycasts pour détecter les murs du circuit :

```

float[] GetSensorData()
{
    float[] sensors = new float[5];
    float[] angles = { -60f, -30f, 0f, 30f, 60f };

    for (int i = 0; i < 5; i++)
    {
        Vector3 dir = Quaternion.Euler(0, angles[i], 0) * transform.forward;
        RaycastHit hit;
        if (Physics.Raycast(transform.position, dir, out hit, 10f))
            sensors[i] = hit.distance / 10f; // Normalisé 0-1
        else
            sensors[i] = 1f; // Rien détecté
    }
}

```

```

    return sensors;
}

```

3. Connecter le Cerveau à la Voiture

On récupère les sensors, on les passe au réseau, et on applique les outputs :

```

void Update()
{
    float[] inputs = GetSensorData();
    float[] outputs = brain.FeedForward(inputs);

    float throttle = outputs[0]; // -1 à 1
    float steering = outputs[1]; // -1 à 1

    // Appliquer au RigidBody
    rb.AddForce(transform.forward * throttle * motorForce);
    rb.AddTorque(transform.up * steering * steerForce);
}

```

4. L'Algorithme Génétique

On gère une population de voitures :

```

void NextGeneration()
{
    // Trier par fitness (distance parcourue)
    population.Sort((a, b) => b.fitness.CompareTo(a.fitness));

    // Garder les meilleures (ex: top 20%)
    int survivors = population.Count / 5;

    // Créer la nouvelle génération
    for (int i = 0; i < population.Count; i++)
    {
        int parentIndex = i % survivors;
        NeuralNetwork child = population[parentIndex].brain.Copy();
        child.Mutate(0.1f); // 10% de mutation
        population[i].brain = child;
        population[i].fitness = 0;
    }
}

```

🧠 5. Accélérer l'Entraînement

L'entraînement peut être très lent en temps réel. Pour l'accélérer :

```
Time.timeScale = 5f; // 5x plus vite
```

On peut aussi désactiver le rendu visuel et ne garder que la physique pour aller encore plus vite.

Astuce : Si la voiture ne progresse pas après plusieurs générations, augmentez le taux de mutation (ex: 0.2) pour explorer de nouvelles stratégies.